Szegedi Tudományegyetem
Informatikai Intézet

SZAKDOLGOZAT

Szabó Bence

2026

Szegedi Tudományegyetem
Informatikai Intézet

DRS Zóna - Motorsport tematikájú közösségi oldal

Szakdolgozat

Készítette:

Szabó Bence
programtervező informatikus
szakos hallgató

Témavezető:

Dr. Bilicki Vilmos
egyetemi adjunktus

Szeged
2026

Feladatkiírás

A jelenlegi motorsport-tematikájú weboldalak fragmentáltak: a hírek, eredmények és közösségi interakció különböző platformokon érhető el, sokszor többszöri regisztrációt követően. Ennek következménye az alacsony felhasználói elköteleződés, az egységes közösségi élmény hiánya, valamint az újságírói szemléletek és a rajongói aktivitás szét szórtsága.

A feladat egy olyan webalkalmazás készítése Angular alapú keretrendszer használatával, ahol a felhasználók egységesen megtalálják a motorsporttal kapcsolatos híreket, eredményeket, valamint aktívan kommunikálhatnak egymással kommentek és szavazások formájában.

Így ez egy centralizált közösségi platform, ahol az újságírók könnyedén hozhatnak létre tartalmakat, a rajongók olvashatnak híreket, követhetik az eredményeket, kommentelhetnek és szavazhatnak - mindezt egyetlen, modern, mobilbarát felületen.

Tartalmi összefoglaló

Téma megnevezése:

DRS Zóna - Motorsport tematikájú közösségi oldal

A megadott feladat megfogalmazása:

Egy motorsport-tematikájú, centralizált közösségi platform, ahol az újságírók könnyedén hozhatnak létre tartalmakat, a rajongók olvashatnak híreket, követhetik az eredményeket, kommentelhetnek és szavazhatnak - mindezt egyetlen, modern, mobilbarát felületen.

Megoldási mód:

A feladatot megoldásához Angular keretrendszert fogok használni TypeScript programozási nyelvvel, mely egy ASP.NET Core (.NET 9) alapokon megvalósuló szerveroldallal kommunikál. REST API-n keresztül kezeli az üzleti logikát és az adatok továbbítását. Az adatbázis-kezelést Entity Framework Core támogatja, amely kapcsolatot biztosít a backend és a PostgreSQL adatbázis között

Alkalmazott eszközök, módszerek:

A fejlesztés során a **JetBrains WebStorm** fejlesztőkörnyezetet használom a frontend elkészítéséhez. A backend megvalósítása **JetBrains Rider** környezetben készül, az adatbázis kezelésére és adminisztrációjára a **DBeaver** adatbázis-kezelő szoftvert alkalmazom, amely a PostgreSQL adatbázis kezelését támogatja.

Elért eredmények:

A webalkalmazást CloudFlare Tunneling segítségével egy külső eszközről hostoltam, valamint a szoftver forráskódját publikáltam GitHub-ra.

A webalkalmazás a reszponzivitása asztali gépen, laptopon, tableten és mobilon is kényelmesen és gyorsan használható, ezáltal egy szélesebb csoportnak nyújt kellemes felhasználói élményt.

Kulcsszavak:

motorsport, közösségi oldal, Angular, ASP.NET, PostgreSQL, drs-zóna

Tartalomjegyzék (nem frissített)

FELADATKIÍRÁS 1
TARTALMI ÖSSZEFOGLALÓ 2
TARTALOMJEGYZÉK 3
BEVEZETÉS 5
1. TERÜLETI ÁTTEKINTÉS 7
1.1. Prezi 7
1.2. Padlet 8
1.3. Wakelet 8
1.4. GoConqr 8
2. FUNKCIONÁLIS SPECIFIKÁCIÓ 9
2.1. Vendég felhasználó 10
2.2. Regisztrált felhasználó 10
3. TERVEZETT MEGJELENÉS 12
3.1. Főoldal 12
3.2. Regisztráció, bejelentkezés 13
3.3. Jegyzetek oldal 14
3.4. Jegyzet részletes oldal 15
3.5. Profilok oldal 15
3.6. Profil részletes oldal 16
3.7. Profilom oldal 17
3.8. Jegyzeteim oldal 17
3.9. Egyéb tervek 19
4. FELHASZNÁLT TECHNOLÓGIÁK 20
4.1. Angular 20
4.2. Angular Material 20
4.3. Google Firebase 20
4.4. Tailwind 20
4.5. Quill 21
4.6. Phosphor Icons 21
4.7. Toaster 21
4.8. Skeleton loader 21
4.9. Angular Fire 21

4.10. Gemnini Ai 21
5. ARCHITEKTÚRA 22
5.1. Frontend 22
5.2. Backend 22
6. BELSŐ FELÉPÍTÉS 23
6.1. Komponensek 23
6.2. Szolgáltatások 27
6.3. Interfészek 28
6.4. Direktívák 29
6.5. Resolverek 30
6.6. Pipeok 30
6.7. Routing 31
7. BIZTONSÁG 32
8. ADATMODELL 34
8.1. Címke 34
8.2. Értesítés 35
8.3. Értékelés 35
8.4. Felhasználó 35
8.5. Hozzászólás 36
8.6. Jegyzet 36
8.7. Módosítási kérés 37
9. A RENDSZER MAGASSZINTŰ FOLYAMATAI 38
10. FONTOSABB KÓDRÉSZLETEK 39
10.1. Regisztráció 39
10.2. Jegyzetek létrehozása, módosítása 40
10.3. Értékelés létrehozása 42
11. TAPASZTALATOK, TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK 44
IRODALOMJEGYZÉK 46
NYILATKOZAT 47
KÖSZÖNETNYILVÁNÍTÁS 48
ELEKTRONIKUS MELLÉKLET 49
MELLÉKLETEK 50

Bevezetés

A digitális technológiák fejlődésével a sporttal kapcsolatos tartalmak fogyasztása jelentősen átalakult. Napjainkban a rajongók már nem csupán televízión vagy hagyományos hírportálokon keresztül követik kedvenc sportágaikat, hanem online felületeken, közösségi médián és mobilalkalmazásokon keresztül is folyamatosan informálódnak. Különösen igaz ez a motorsport világára, ahol a versenyek gyors tempója, az állandó technikai fejlesztések, a pilóták közötti rivalizálás és a folyamatos hírek kiemelt érdeklődést váltanak ki a szurkolókból.

A motorsport rajongói gyakran több különböző platformot kénytelenek használni annak érdekében, hogy naprakész információkhoz jussanak. Egyes oldalakon híreket olvashatnak, máshol eredményeket követhetnek, külön felületeken kommentelhetnek véleményt nyilváníthatnak, formálhatnak véleményt. Ez a széttagoltság kényelmetlen felhasználói élményt eredményez, valamint megnehezíti egy valódi online közösség kialakulását.

A szakdolgozat célja egy olyan központi, modern webes platform megtervezése és fejlesztése, amely egyetlen rendszerben egyesíti a motorsporttal kapcsolatos legfontosabb funkciókat. A felhasználók híreket olvashatnak, követhetik az aktuális eredményeket, hozzászólhatnak a tartalmakhoz, valamint szavazásokon vehetnek részt. Emellett az újságírók és tartalomkészítők számára is egyszerűen kezelhető felület biztosított új cikkek és bejegyzések létrehozására.

A rendszer fejlesztése során kiemelt szempont volt a korszerű technológiai háttér alkalmazása, a gyors működés, a reszponzív megjelenés és a későbbi bővíthetőség. Ennek érdekében olyan technológiákkal dolgoztam, amelyekkel a jelenkor megoldásait egyszerűbben meg lehet oldani.

A dolgozat bemutatja a rendszer tervezési folyamatait, az alkalmazott technológiákat, az adatbázis felépítését, valamint a megvalósítás során használt módszereket és eredményeket

1. Piackutatás

A fejlesztés megkezdése előtt fontos volt megvizsgálni a hazai motorsporttal foglalkozó online platformokat annak érdekében, hogy feltérképezhető legyen a jelenlegi kínálat, valamint azonosíthatók legyenek azok a hiányosságok, amelyekre az elkészítendő rendszer megoldást nyújthat.

1.1 Formula.hu

A Formula.hu az egyik legismertebb hazai motorsporttal foglalkozó hírportál, amely elsősorban Forma–1-es és egyéb autósport hírekkel foglalkozik. Az oldal erőssége a rendszeresen frissülő tartalom, valamint a részletes cikkek és elemzések. Emellett több motorsport kategóriát is lefed, így széles célközönséget ér el.

Hátrányként említhető, hogy a közösségi funkciók kevésbé hangsúlyosak. A kommentelési lehetőségek korlátozottak, és a felhasználói aktivitást ösztönző elemek – például szavazások vagy interaktív közösségi funkciók – kevésbé jelennek meg.

1.2 Motorsport.hu

A Motorsport.com nemzetközi háttérrel rendelkező oldal, amely magyar nyelven is elérhető. Előnye a professzionális megjelenés, a gyors hírszolgáltatás, valamint a multimédiás tartalmak – például képgalériák és videók – használata. Széleskörűen foglalkozik különböző motorsport sorozatokkal.

Ugyanakkor a platform inkább hírportál jellegű, mint közösségi oldal. A felhasználók közötti interakció alacsonyabb szintű, ezért kevésbé alkalmas aktív rajongói közösség kialakítására.

1.3 Motorsportol.hu

A Motorsportol.hu egy modernebb megjelenésű hazai motorsport oldal, amely friss hírekkel és aktuális tartalmakkal várja az érdeklődőket. Külön előnye, hogy bizonyos interaktív funkciók, például szavazások is megtalálhatók rajta, amelyek növelik a felhasználói bevonódást.

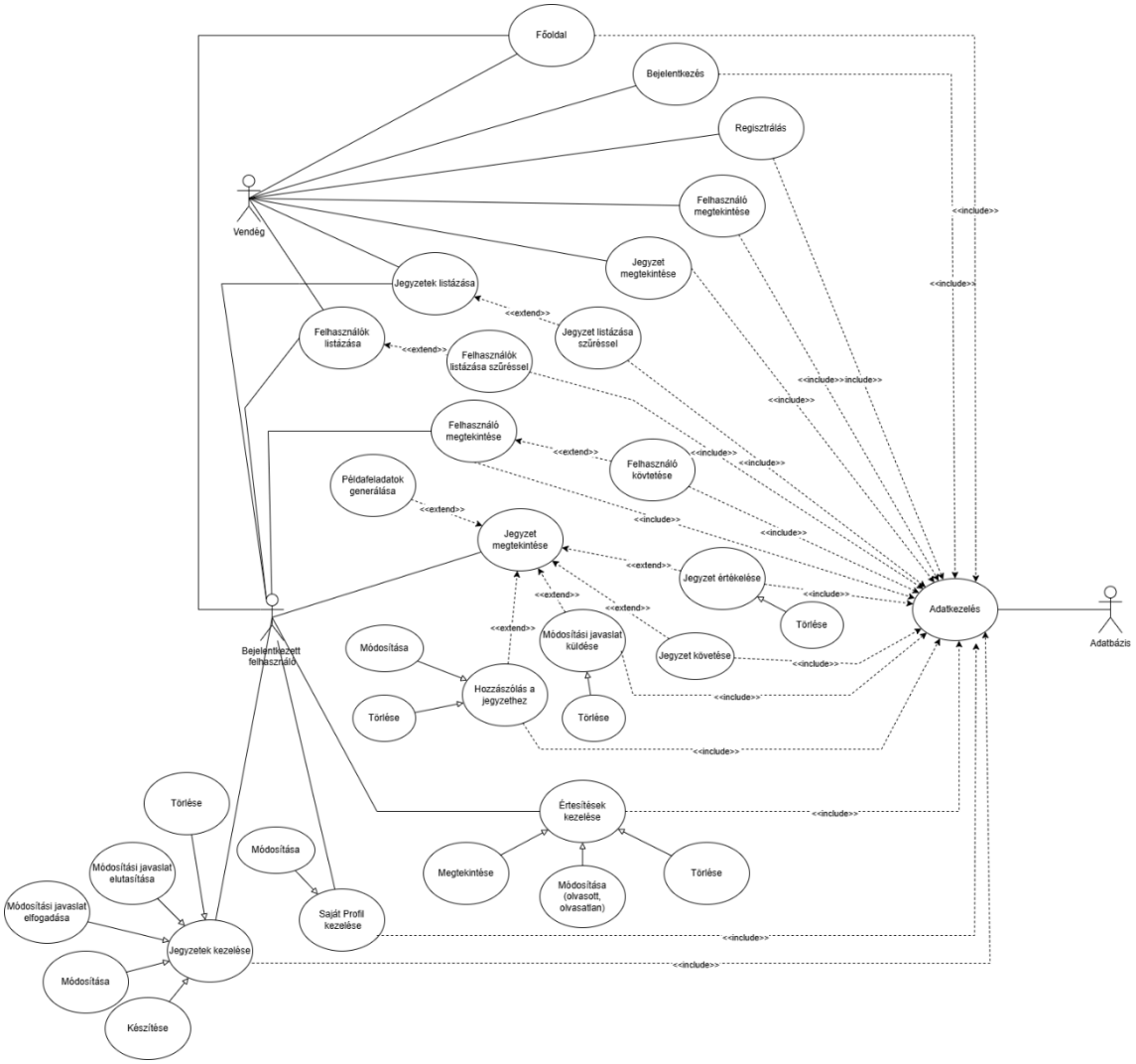
Hátránya azonban, hogy nincs lehetőség kommentelésre, így a látogatók véleménycserére vagy közvetlen kommunikációra kevésbé tudják használni a felületet.

1.4 Összegzés

A piackutatás alapján megállapítható, hogy a jelenlegi hazai motorsport oldalak elsősorban hírszolgáltatásra koncentrálnak, azonban a közösségi funkciók sok esetben hiányosak vagy korlátozottak. Ez indokolja egy olyan modern platform fejlesztését, amely ötvözi a naprakész hírtartalmat az aktív közösségi lehetőségekkel, például kommenteléssel, szavazással és felhasználói interakciókkal.

2. Funkcionális specifikáció (nem frissített)

A használati eset diagram segítségével a rendszerünk által elérhető funkcionalitásokat és az azokat használó aktorok közötti kapcsolatokat modellezhetjük le a legmagasabb absztrakciós fokon. Az aktorok a felhasználók, akik valamilyen módon interakcióba lépnek a rendszerrel, míg a használati esetek maguk a rendszer által kínált funkciók. Ezek a diagramok mutatják be a rendszer tervezett funkcióit, a rendszer környezetét és ezek kapcsolatait.



2.1. ábra: A webalkalmazáshoz készült használati eset diagram

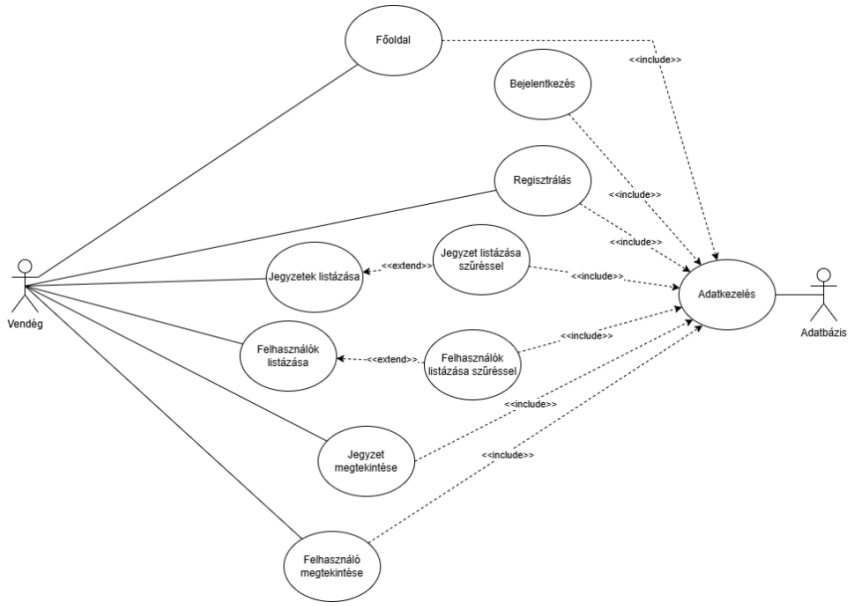
A használati eset diagramok jelölésrendszere az UML része. Az aktorokat, felhasználókat pálcikaemberrel, a rendszer funkcióit oválissal, az aktorok és funkciók közötti kapcsolatokat különböző vonalakkal jelöljük.

A webalkalmazás felhasználóit két csoportba sorolhatjuk: vendég felhasználók és regisztrált fiókkal rendelkező felhasználók. A két csoport közötti különbség jelentős, hiszen a regisztrált fiókkal rendelkező felhasználók sokkal több funkciót érnek el, mint az oldalra bejelentkezés nélkül látogató felhasználók.

2.2. Vendég felhasználó

A vendég felhasználóknak a webalkalmazásban lehetőségük van regisztrálni, illetve amennyiben van már regisztrált fiókjuk, lehetőségük van belépni abba.

Ha továbbra is vendég felhasználóként szeretnék használni a webalkalmazást, akkor a lehetőségek száma lekorlátozódik. Lehetőségük van a jegyzeteket böngészni, listázni szimplán, valamint a szűrés funkciók használatával is. Emellett képesek a jegyzeteket és felhasználói profilokat megtekinteni, valamint a főoldalra ellátogatni.



2.2.1. ábra: Használati eset diagram a vendég aktorral és a hozzá tartozó funkciókkal.

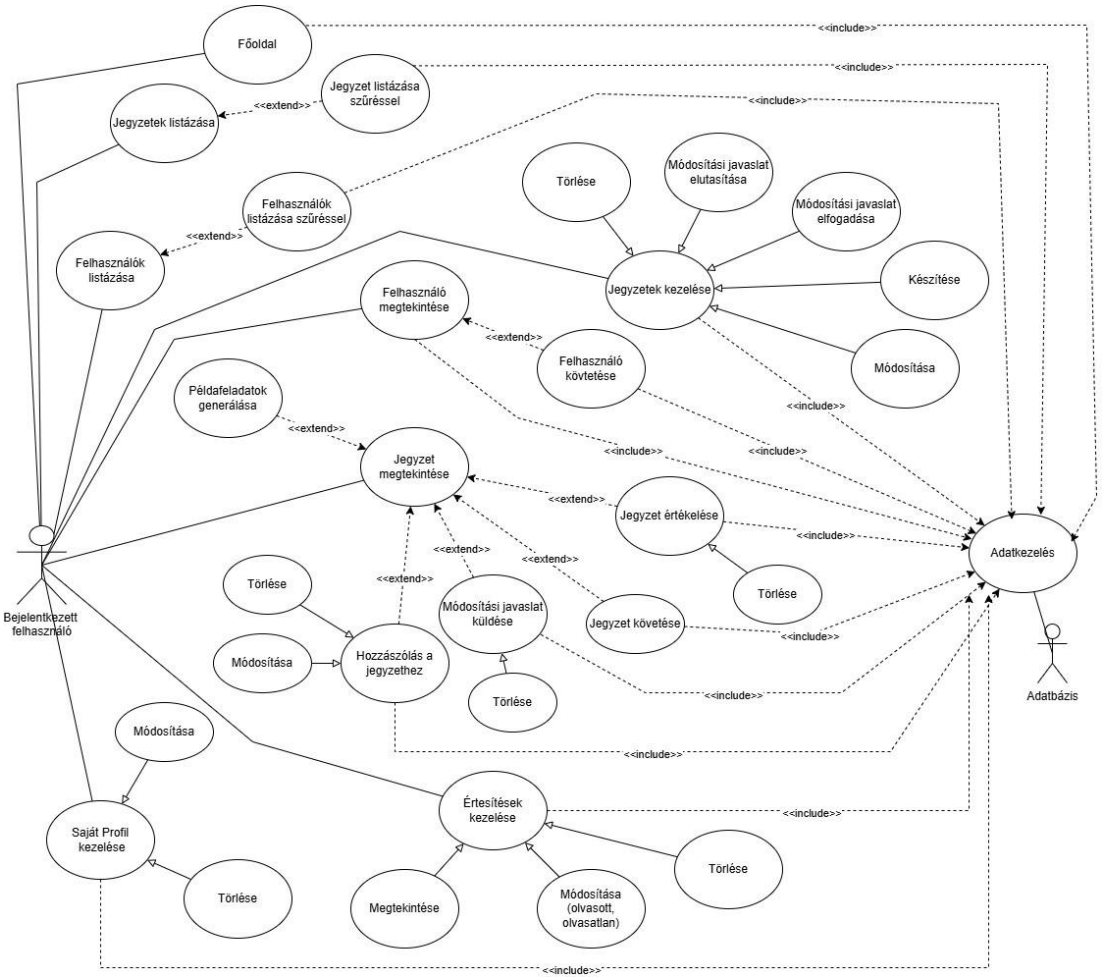
2.3. Regisztrált felhasználó

A vendég felhasználókhoz képest a bejelentkezett felhasználók korlátozás nélkül hozzáférnek a webalkalmazás összes funkcionalitásához.

A bejelentkezés után a felhasználók a főoldalra kerülnek. Ezután a jegyzeteket a vendég felhasználókhoz hasonlóan tudják listázni szimplán, de akár szűréssel is. A jegyzeteket is meg tudják tekinteni a vendég felhasználókhoz hasonlóan, viszont a megtekintés mellett van lehetőségük a jegyzetekhez hozzászólást írni, módosítási javaslatot fűzni, értékelést küldeni, valamint példafeladatokat is generálhatnak a jegyzethez. Ezek mellett a bejelentkezett felhasználók tudnak jegyzeteket is írni, a jegyzeteiket módosítani, törölni, illetve a módosítási kéréseket kezelni: elfogadni, elutasítani.

A bejelentkezett felhasználók a profiljukat testre tudják szabni, valamint a többi felhasználó profilját is meg tudják tekinteni.

A belépett felhasználóknak megnyílik a követés funkció is: be tudnak követni jegyzeteket és felhasználókat is.



2.3.1. ábra: Használati eset diagram fiókkal rendelkező aktorral és a hozzá tartozó funkcionalitásokkal.

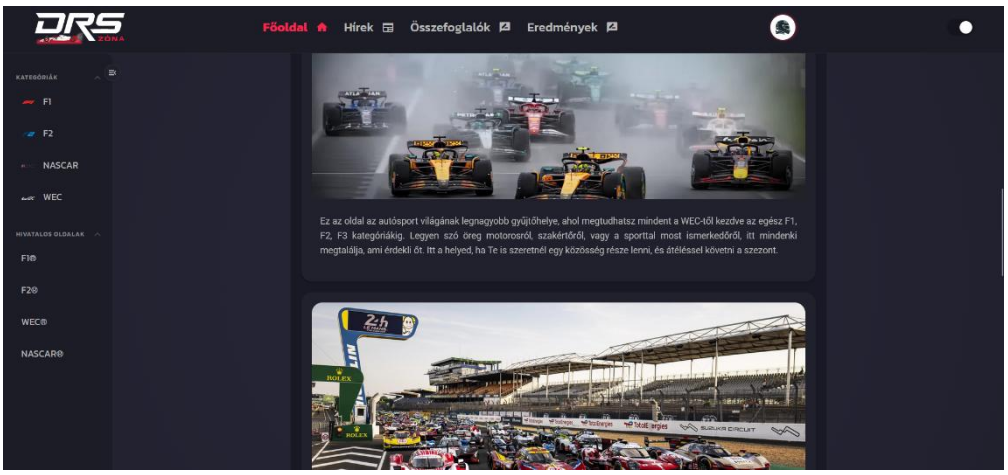
3. Tervezett megjelenés

A webalkalmazáshoz tartozó megjelenés egy egyetemi projekt keretein belül elkészült egyszerűbb webalkalmazás továbbfejlesztett változata. A motorsport világához kellően illő sötét, vörös szín, valamint a melegebb fehér és szürke megjelenés dominált.

A webalkalmazásnál nagy hangsúly volt a reszponzivitás-on, hogy a felhasználók bármilyen eszközön könnyen és egyszerűen tudjanak tanulni. Fejlesztés során ezért több különböző mobil, illetve tablet képernyőjének mérete is figyelembe lett véve. Az Angular stílusokra több eszközt is felajánl, én az SCSS-t választottam. Ez lényegében a közismertebb CSS StyleSheet egy tovább gondolása, ahol saját változókat lehet előre definiálni, megkönnyítve több ugyanolyan komponens kinézetének módosítását.

3.1 Főoldal

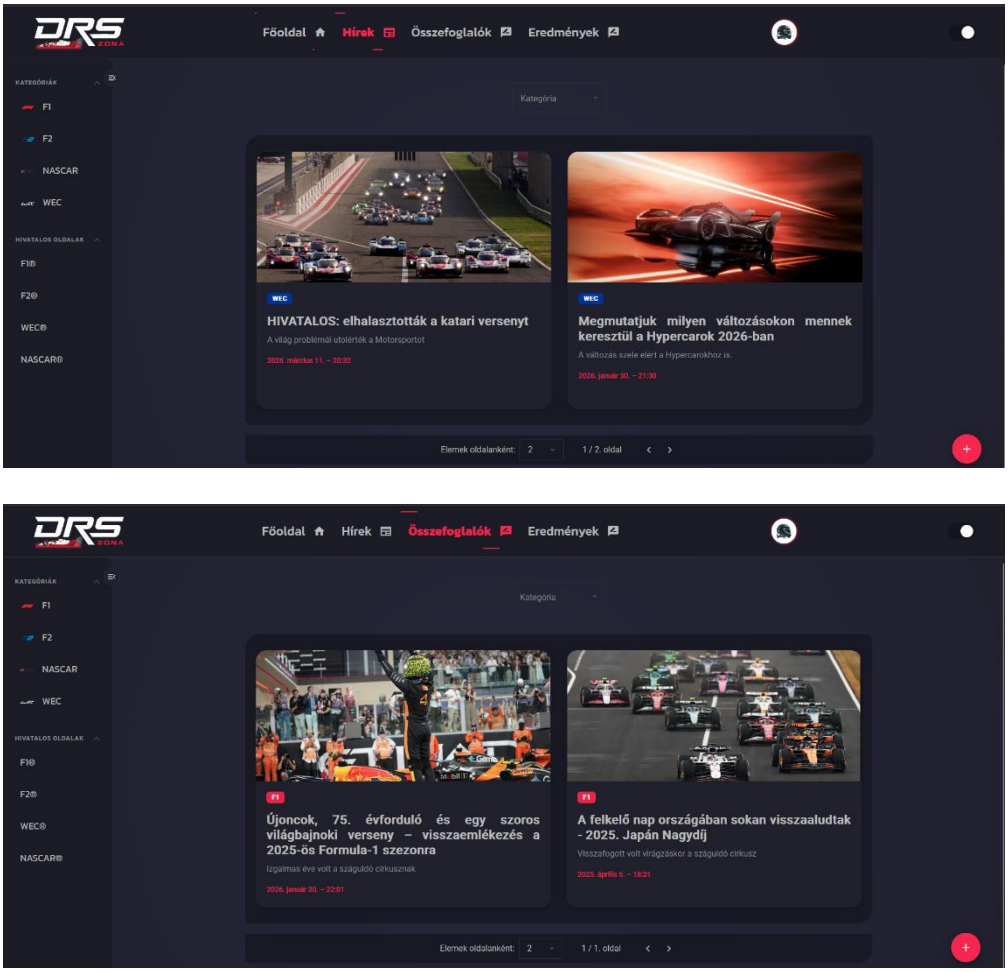
A webalkalmazás látogatása során először a főoldalra navigálunk. Itt megtalálhatóak az az Aktuális Szavazások, a Legfrissebb Hírek, illetve az oldal rövid, egyszerű leírása. Az oldal tetején megtalálható a navigáció, a Bejelentkezés, illetve a téma váltó gomb.



3.1.1. ábra: A webalkalmazás főoldalának asztali nézete sötét módban

3.2 Hírek, Összefoglalók oldal

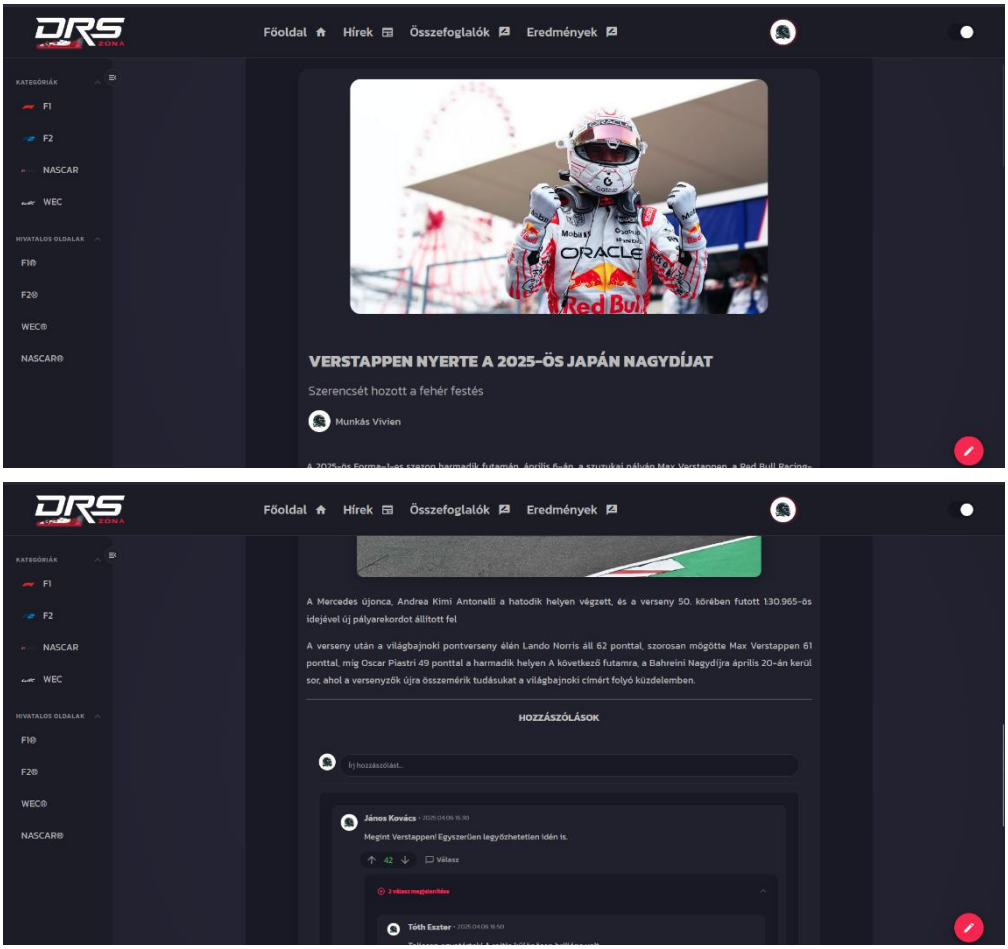
Ezeken az oldalakon megtalálhatóak hírek és összefoglalók, ahol listázást kategória szerint lehet szűrni, valamint a beépített Angular Material könyvtár segítségével az oldalazés is támogatott. Az *Admin* és *Writer* role-al rendelkezőknek megjenik továbbá egy létrehozás gomb.



3.2.1 ábra: A webalkalmazás hír és összefoglaló oldalainak asztali nézete sötét módba

3.3 Hír oldal

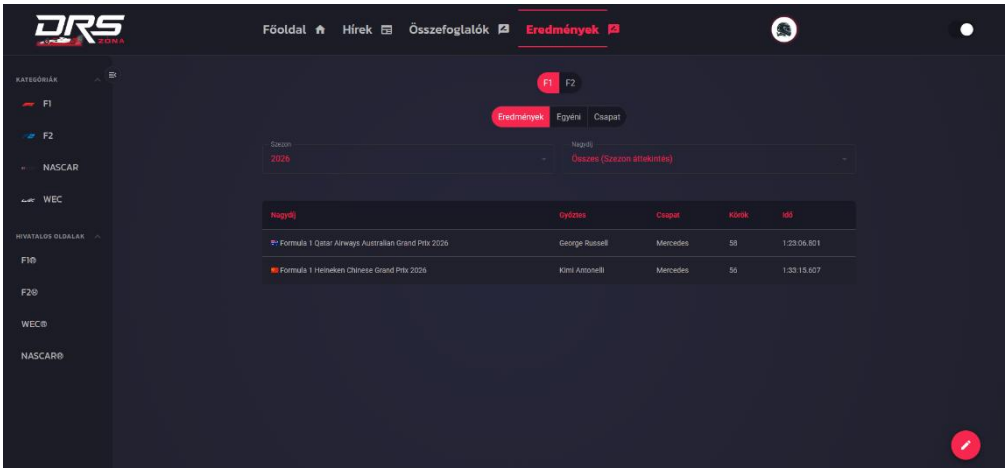
Az oldal tartalmazz a 3.2-es pontban tárgyalt Hír oldal bármelyik kártyájának tartalmát. Az oldalon a hír további tartalma található meg: cím, lead szöveg, szerző profilképe, neve, valamint a hír tartalma. Emellett az oldal alján megjelenik a hírhez tartozó Hozzászólás szekció.



3.3.1 ábra: A Hír oldal és az ahhoz tartozó kommentszekció asztali nézete sötét módban

3.4 Eredmények

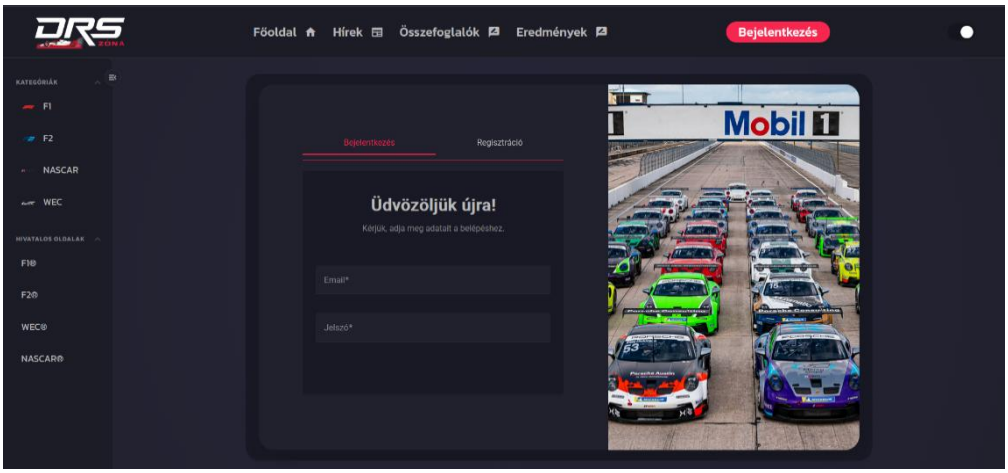
Az eredmények oldalon megtalálható az F1 és F2 sportágak eredményei. Sportág, Esemény, Nagydíj, és Nagydíj eseményen szűrhetők az eredmények. Emellett az Egyéni és Csapat (konstruktőri) bajnokság is elérhető. Mind a bajnokság, mind az eredmények nézhetők egy esemény, pilóta, csapat szerint.



3.4.1 ábra: A webalkalmazás Eredmények oldalának asztali nézete sötét módban

3.5 Bejelentkezés, Regisztráció

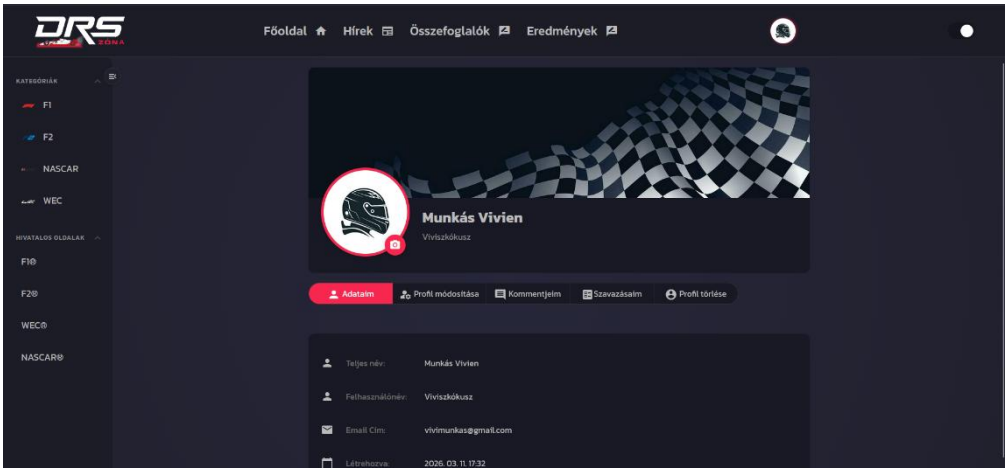
Ezeken az oldalakon megtalálhatóak hírek és összefoglalók, ahol listázást kategória szerint lehet szűrni, valamint a beépített Angular Material könyvtár segítségével az oldalazés is támogatott. Az *Admin* és *Writer* role-al rendelkezőknek a 3.4.1-es ábrán megjelenik továbbá egy létrehozás gomb.



3.5.1 ábra: A Bejelentkezés és Regisztrációoldal asztali nézete sötét módban

3.6 Profil oldal

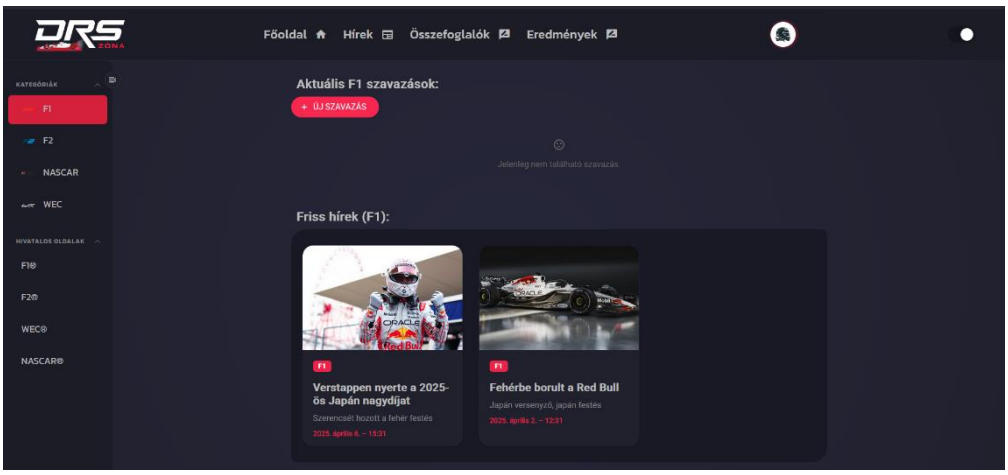
Ezeken az oldalakon megtalálhatóak hírek és összefoglalók, ahol listázást kategória szerint lehet szűrni, valamint a beépített Angular Material könyvtár segítségével az oldalazés is támogatott. Az *Admin* és *Writer* role-al rendelkezőknek a 3.4.1-es ábrán megjelenik tovább egy létrehozás gomb.



3.6.1 ábra: A Bejelentkezés és Regisztrációoldal asztali nézete sötét módban

3.7 Kategória oldal

A kategória oldal felépítése szinte teljesen ugyanaz a már korábban kifejtett 3.1-es oldalhoz. A fontos különbség az, hogy a szavazások, a hírek, valamint az rövid leírás már az adatbázisban tárolt szériához tartozik.



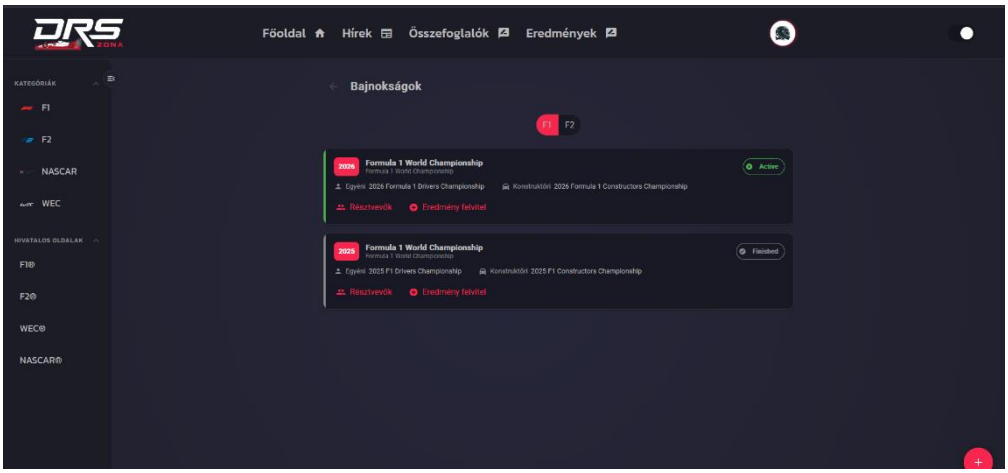
3.7.1 ábra: A Bejelentkezés és Regisztrációoldal asztali nézete sötét módban

3.8 Adminisztrációs oldalak

A fejezetben a rendszer legfontosabb és funkcionálisan legjellemzőbb felületei kerülnek bemutatásra. Az alkalmazás a bemutatott képernyőkön túl további adminisztrációs, szerkesztési és adatkezelési nézeteket is tartalmaz, azonban a terjedelmi korlátok miatt csak a főbb oldalak és a működés szempontjából meghatározó felületek kerülnek részletes ismertetésre.

3.8.1 Bajnokságok kezelése

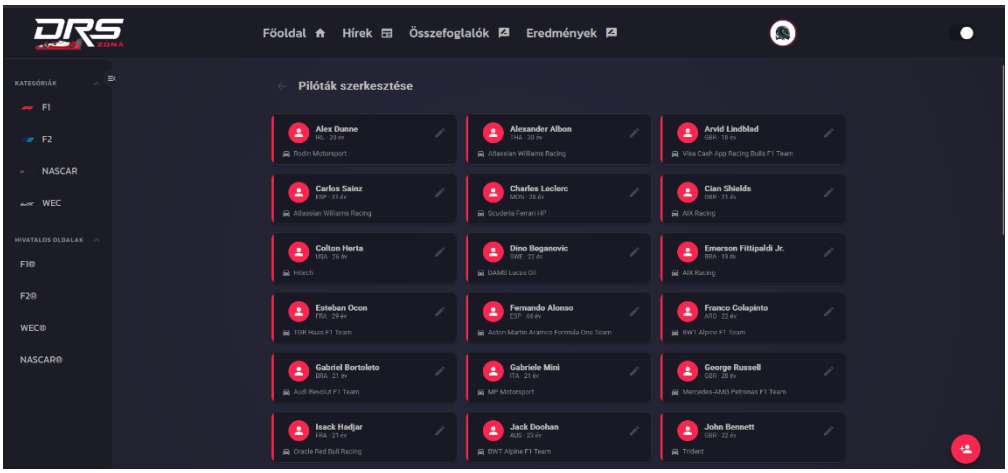
A bajnokságok oldalon az adatbázisban létrehozott összes bajnokságot. Minden bajnokságnak a résztvevőit meg lehet külön tekinteni, valamint egy adott bajnoksághoz tartoznak konkrét eredmények is. A plusz gombra kattintva új bajnoksághoz plusz bajnokságot is létrehozhatunk.



3.8.1.1 ábra: A Bajnokságok kezelése oldal asztali nézete sötét módban

3.8.2 Pilóták kezelése

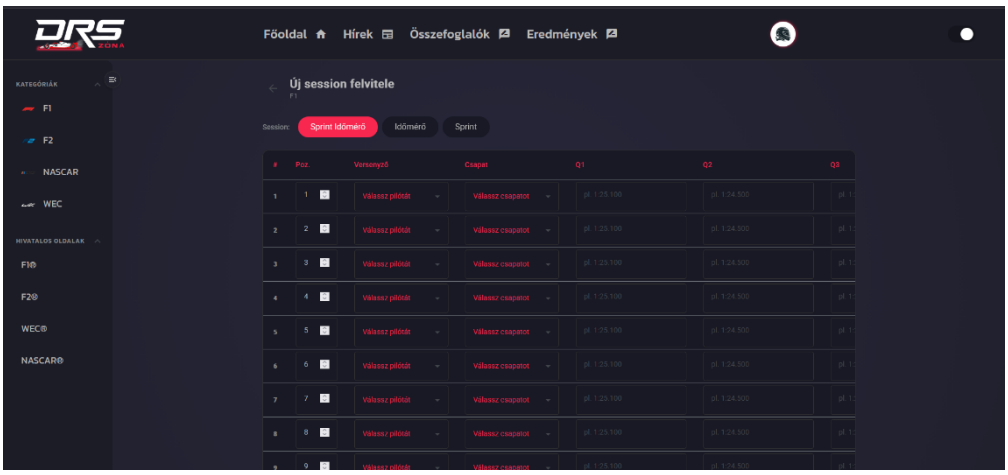
A pilótákat táblázatos formában listázzuk ki. Minden pilótának adata szerkeszthető, amely szerkesztés után dinamikusan változik. A plusz gombra kattintva új pilótát hozhatunk létre.



3.8.2.1 ábra: A Pilóták kezelése oldal asztali nézete sötét módban

3.8.3 Futameredmények rögzítése

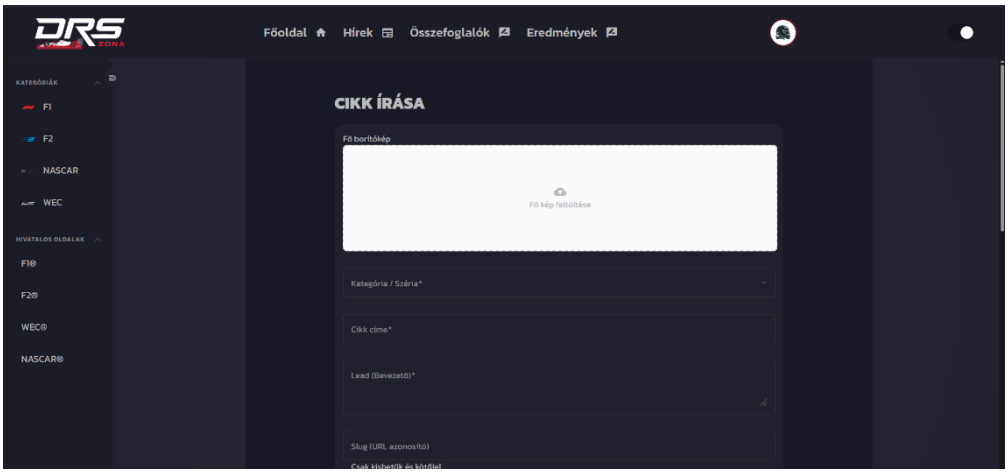
Minden bajnoksághoz futamok tartoznak, a futamokhoz pedig eredmények. Sportág szerint minden eredményt máshogy visz be a rendszer. Figyeli milyen eseményhez írunk eredményeket, a pontrendszerre sportág szerint, valamint arra, hogy melyik pilóta melyik csapaté, így könnyítve a felhasználói interakciót. A táblázat dinamikusan változik eseménytől függően. A felhasználónak verseny esetén az első helyezett teljes idejét, valamint a többi pilóta távolságát ezredmásodpercig (pl. 0.242s).



3.8.3.1 ábra: A Futameredmények kezelése oldal asztali nézete sötét módban

3.8.4 Cikkek létrehozása

Az újságírók egy CMS szerű felületet látnak cikk íráskor. A cikk szekciói emellett külön vannak osztva hír és összefoglaló szerint. A WYSIWYG felületet egy külső TypeScript-re írt a kolkov/angular-editor csomag biztosítja



3.8.4.1 ábra: A Cikkek létrehozása oldal asztali nézete sötét módban

Összegzés

A bemutatott képernyők mellett a rendszer további adminisztrációs aloldalakat, létrehozó és szerkesztő dialógusokat, valamint jogosultsághoz kötött funkciókat is tartalmaz.

4. Felhasznált technológiák

4.1. Frontend fejlesztés

4.1.1. Angular

Az Angular egy nyílt forráskódú, TypeScript alapú keretrendszer. Az Angular elsősorban frontend oldali fejlesztéshez, összetett webes alkalmazások készítéséhez használják. Komponensalapú architektúrája támogatja a moduláris fejlesztést, az újrafelhasználhatóságot és a skálázhatóságot.

4.1.2. Angular Material

Az Angular Material az Angular UI komponens könyvtára, amely kiváló minőségű, könnyen hozzáférhető és kezelhető komponenseket biztosít a Material dizájn irányelveknek megfelelően.

4.1.3. TypeScript

Az alkalmazás frontend része TypeScript nyelven készült, amely a JavaScript kibővített, típusos változata. Használata növeli a kód megbízhatóságát és karbantarthatóságát.

4.1.4. SCSS

A stíluslapok elkészítéséhez SCSS került felhasználásra, amely lehetővé teszi változók, beágyazott szabályok és újra felhasználható stíluselemek alkalmazását.

4.1.5. @kolkov/angular-editor

A cikkek szerkesztéséhez egy nyílt forráskódú text editor került beépítésre a @kolkov/angular-editor könyvtár segítségével. Célja a WYSIWYG (What You See Is What You Get) elv alkalmazása, mely lehetővé teszi az egyszerűbb cikk írását.

4.2. Backend Fejlesztés

4.2.1. ASP.NET Core

A szerveroldali alkalmazás ASP.NET Core keretrendszerben készült, amely REST API szolgáltatások létrehozását és üzleti logika megvalósítását tette lehetővé.

4.2.2. Entity Framework Core

Az adatkezeléshez Entity Framework Core ORM került felhasználásra, amely objektumorientált módon kapcsolja össze az alkalmazást a relációs (SQL alapú) adatbázissal.

4.2.3. JWT autentikáció

A felhasználók hitelesítéséhez JSON Web Token (JWT) alapú autentikáció került kialakításra. Ennek segítségével biztonságosan kezelhető a bejelentkezett felhasználók jogosultsága és munkamenete.

4.2.4. Swagger

Az elkészült REST API dokumentálására és tesztelésére Swagger került alkalmazásra, amely interaktív felületet biztosít a végpontok tesztelésére, és a séma leírásására. Segítségével emellett az előállított OpenAPI dokumentáció szolgált alapul az ng-openapi kliensgenerátor számára, amely automatikusan elkészítette a frontend oldali API hívásokhoz szükséges TypeScript interfészeket és szervizeket.

4.3. Adatbázis

4.3.1. PostgreSQL

Az alkalmazás relációs adatbázisaként PostgreSQL került kiválasztásra, amely megbízható, stabil és széles körben alkalmazott nyílt forráskódú adatbázis-kezelő rendszer.

4.3.2. DBeaver

Az adatbázis kezelésére és adminisztrációjára a DBeaver kliensprogram szolgált.

4.4. Fejlesztői eszközök

4.4.1. JetBrains WebStorm

A frontend fejlesztéshez JetBrains WebStorm fejlesztői környezet került használatra, amely hatékonyan kezelte az Angular változóit és struktúráját

4.4.2. JetBrains Rider

A backend fejlesztéshez JetBrains Rider integrált fejlesztői környezet került alkalmazásra.

4.4.3. Git és GitHub

A verziókövetés Git rendszerrel történt, míg a forráskód tárolására és mentésére GitHub szolgált.

4.5 Infrastruktúrális megoldások

1.1.1. Cloudflare Tunnel

Az alkalmazás külső elérésének biztosítására Cloudflare Tunnel került alkalmazásra. A megoldás lehetővé teszi, hogy a helyi vagy belső hálózaton futó alkalmazás biztonságosan, nyilvános portnyitás nélkül legyen elérhető az internet felől.

1.1.2. Domain

Az alkalmazás eléréséhez saját domain név került konfigurálásra, amely egyszerűbb hozzáférést és professzionálisabb megjelenést biztosít a rendszer számára.

5. Architektúra

Az elkészített webalkalmazás többrétegű kliens–szerver architektúrára épül. Ennek célja a rendszer elemeinek logikai elkülönítése, a könnyebb karbantarthatóság és a későbbi bővíthetőség biztosítása. A rendszer három fő komponensből áll: frontend kliensalkalmazásból, backend API szolgáltatásból, valamint adatbázis rétegből.

A frontend réteg biztosítja a felhasználói felületet, ahol a látogatók megtekinthetik a híreket, eredményeket, statisztikákat, illetve a jogosultsággal rendelkező felhasználók adminisztrációs műveleteket végezhetnek. A kliensoldali alkalmazás Angular keretrendszerben készült.

A backend komponens ASP.NET Core alapú REST API szolgáltatásként valósult meg. Feladata a kliensoldalról érkező HTTP kérések fogadása, feldolgozása, az üzleti logika végrehajtása, valamint az adatbázissal történő kommunikáció kezelése.

Az adatbázis réteg PostgreSQL alapokon működik, amely az alkalmazás tartós adatainak tárolását végzi. Itt kerülnek eltárolásra többek között a felhasználók, cikkek, kommentek, szavazások, bajnokságok, pilóták, csapatok és futameredmények adatai.

A komponensek közötti kommunikáció HTTP protokollon keresztül történik. A frontend JSON formátumú kéréseket küld a backend API részére, amely feldolgozza azokat, majd JSON válasz formájában küldi vissza az eredményt. A backend az adatbázissal Entity Framework Core segítségével kommunikál.

5.1. Frontend

5.1.1. Angular SPA

A frontend réteg Angular keretrendszer használatával készült egyoldalas alkalmazásként (Single Page Application). Ennek előnye, hogy az oldalak közötti navigáció teljes újratöltés nélkül történik, ezáltal gyorsabb és folyamatosabb felhasználói élményt biztosít.

5.1.2. Komponensalapú működés

Az alkalmazás felépítése komponensalapú. Az egyes oldalak és funkcionális egységek külön komponensekben kerültek megvalósításra, például főoldal, híroldal, eredmények, profiloldal vagy adminisztrációs nézetek.

5.1.3. Routing rendszer

Az oldalak közötti navigáció Angular Router segítségével valósul meg. A route rendszer támogatja a dinamikus paraméterezést, valamint az útvonalvédelmet is.

5.1.4. Jogosultságkezelés

A védett oldalak elérését route guardok biztosítják. A rendszer külön kezeli a vendég, bejelentkezett, szerzői, szerkesztői és adminisztrátori jogosultságokat.

5.1.5. API kommunikáció

A backend szolgáltatások eléréséhez a frontend HTTP kliensen keresztül kommunikál a REST API-val. Az API hívásokhoz szükséges TypeScript interfészek és service osztályok automatikus generálása ng-openapi segítségével történt az OpenAPI specifikáció alapján.

5.1.6. UI Réteg

A kezelőfelület kialakításához Angular Material komponenskönyvtár került felhasználásra, amely egységes vizuális megjelenést és modern felhasználói élményt biztosít.

5.2 Backend

5.2.1 REST API felépítés

A szerveroldali rendszer REST szemléletű API-ként működik, amely HTTP protokollon keresztül fogadja a kliensalkalmazás kéréseit. A különböző erőforrások - például cikkek, felhasználók, kommentek, bajnokságok vagy eredmények - önálló végpontokon keresztül érhetők el. Az adatok továbbítása JSON formátumban történik.

5.2.2 Controller réteg

A Controller réteg biztosítja az API végpontokat, és fogadja a frontend felől érkező HTTP kéréseket. Feladata a beérkező paraméterek kezelése, az alapvető validációk végrehajtása, majd a megfelelő üzleti logikát megvalósító szolgáltatás meghívása. A beérkező paraméterek úgynevezett DTO-k (Data To Object-ek), amelyek lehetővé teszik a több adatforrásból származó adat

A rendszerben külön kontrollerek kezelik többek között a hitelesítést, a cikkekkel kapcsolatos műveleteket, a kommenteket, a szavazásokat, valamint a motorsport eredményekhez tartozó funkciókat.

5.2.3 Service réteg

A Service réteg tartalmazza az alkalmazás üzleti logikáját. Ebben a rétegben történik a különböző műveletek feldolgozása, a jogosultságellenőrzés, valamint az összetettebb folyamatok kezelése.

Ilyen feladat lehet például cikkek létrehozása és módosítása, felhasználói műveletek kezelése, szavazatok feldolgozása, illetve a futameredmények pontszámításához és rangsorolásához kapcsolódó logika végrehajtása.

5.2.4 Repository réteg

A Repository réteg felelős az adatbázis-műveletek elkülönítéséért. Ez a réteg végzi az adatok lekérdezését, mentését, módosítását és törlését. A repository mintázat alkalmazása elősegíti a tisztább kódszerkezet kialakítását, valamint csökkenti az üzleti logika és az adattárolási megoldás közötti közvetlen kapcsolatot.

5.2.5 Függőséginjektálás

A backend rendszerben a komponensek közötti kapcsolatok kialakítása függőséginjektálás (Dependency Injection) segítségével történik. Ennek eredményeként a kontrollerek és szolgáltatások a szükséges komponenseket automatikusan kapják meg, amely egyszerűsíti a fejlesztést és javítja a tesztelhetőséget.

5.2.6 Hitelesítés és jogosultságkezelés

A védett végpontok elérését JWT token alapú hitelesítés biztosítja. Sikeres bejelentkezés után a rendszer token formájában adja vissza a felhasználó azonosításához szükséges adatokat, amelyet a kliensoldali alkalmazás a későbbi kérések során használ.

A jogosultságkezelés lehetővé teszi a különböző szerepkörök - például általános felhasználó, szerző vagy adminisztrátor - elkülönítését.

5.3 Adatbázis

5.3.1 Főbb adattáblák

Az adatmodell több, egymással kapcsolatban álló táblából épül fel. A legfontosabb entitások közé tartoznak a felhasználók, cikkek, kommentek, szavazások, bajnokságok, pilóták, konstruktőrök, szerződések és futameredmények.

5.3.2 Kapcsolatok és integritás

Az adatmodell több, egymással kapcsolatban álló táblából épül fel. A legfontosabb entitások közé tartoznak a felhasználók, cikkek, kommentek, szavazások, bajnokságok, pilóták, konstruktőrök, szerződések és futameredmények.

5.3.3 Entity Framework Core használata

A backend és az adatbázis közötti kapcsolatot Entity Framework Core biztosítja. Az ORM megoldás lehetővé teszi, hogy az adatbázistáblák objektumorientált módon legyenek kezelhetők. A rendszer támogatja a migrációk használatát is, amelyek segítségével az adatbázisszerkezet verziókövetetten módosítható.

5.3.4 Adminisztráció és fejlesztői eszközök

Az Az adatbázis kezeléséhez és ellenőrzéséhez a fejlesztés során DBeaver kliensalkalmazás került felhasználásra. Az eszköz grafikus felületet biztosít a táblák kezelésére, adatok megtekintésére és SQL lekérdezések futtatására.

6. Belső felépítése (nem frissített)

Az Angular[1] 17-es verzióban az önálló (standalone) komponensek továbbfejlődtek és nagyobb hangsúlyt kaptak. Ennek köszönhetően fel tudjuk építeni az alkalmazásunkat Modulok használata nélkül, ami meggyorsítja és egyszerűsiti a fejlesztést. A standalone komponensek közvetlenül importálhatják a sablonjaikban használt egyéb komponenseket, direktívákat és pipeokat.

6.1. Komponensek (Component)

A webalkalmazás több önálló komponensből épül fel. Ezek a komponensek tartalmazhatnak más önálló komponenseket, direktívákat (directive), pipeokat, szolgáltatások (service) és egyéb kódokat.

A webalkalmazásban található komponenseket két nagy csoportba lehet sorolni. Az egyik csoport a Page komponensek, ide tartoznak azok a komponensek, amiker rá lehet navigálni. A Page komponensek neve minden esetben ’Page’-re végződik. A másik csoportba azok a komponensek vannak, amik a Page komponenseken belül vannak használva, ezek a Pure komponensek. A Jegyzi webalkalmazásban az alábbi Page komponenseket hoztam létre:

- *App*:
  A webalkalmazás gyökér komponense, a RouterOutlet segítségével tölti be dinamikusan az éppen szükséges komponenseket.
- *HomePage*:
  A webalkalmazás főoldala, ez a komponens jelenik meg a webalkalmazás megnyitásakor, a bejelentkezés és a kijelentkezés után.
- *LoginPage*:
  A webalkalmazás bejelentkezés oldala. Itt tudnak a felhasználók a felületre bejelentkezni, és regisztrálni.
- *MyNotesShellPage*:
  A webalkalmazás Jegyzeteim oldalak ’héja’, a menüsávot tartalmazza ami a navigálásért felel a Jegyzeteim oldalak között.

- *MyNotesListPage*:
  A webalkalmazás Jegyzeteim oldalainak a főoldala, ez a komponens töltődik be, amikor a felhasználók a Jegyzeteim menüpontra kattintanak. Ez a komponens felel a felhasználó által készített jegyzetek kilistázásáért.
- *FollowedNotesListPage*:
  A webalkalmazás Jegyzeteim oldalainak a Követett jegyzetek aloldala. Ez a komponens felel a felhasználó által bekövetett jegyzetek kilistázásáért.
- *ModifyRequestsListPage*:
  A webalkalmazás Jegyzeteim oldalainak Módosítási kérések aloldala. Ez a komponens felel a felhasználó által írt módosítási kérések kilistázásáért, illetve a felhasználó által írt jegyzetekre érkezett módosítási kérések kilistázásáért.
- *MyNotesDetailsPage*:
  Ez a komponens felel a felhasználó által írt jegyzet részleteinek a megjelenítéséért. A felhasználók a *MyNotesListPage* illetve a *NoteListPage* komponensekről tudnak idenavigálni.
- *NoteListPage*:
  A webalkalmazás Jegyzetek oldala, a felhasználók a Jegyzetek menüpontra kattintva tudnak idenavigálni. Ez a komponens felel a weboldalra feltöltött jegyzetek kilistázásáért.
- *NoteDetailsPage*:
  Ez a komponens felel a jegyzetek részleteinek a megjelenítéséért. A felhasználók a *NoteListPage* komponensen keresztül tudnak idenavigálni a kiválasztott jegyzet kártyára kattintva. Amennyiben van bejelentkezett felhasználó, és ő a kiválasztott jegyzet szerzője, a Jegyzeteim oldal jegyzet részletes oldala töltődik be.
- *ProfileListPage*:
  A webalkalmazás Felhasználók oldala, a felhasználók a Felhasználók menüpontra kattintva tudnak idenavigálni. Ez a komponens felel a regisztrált felhasználók kilistázásáért. Amennyiben van bejelentkezett felhasználó, az ő kártyája nem jelenik meg a listában.
- *ProfileDetailsPage*:
  Ez a komponens felel a felhasználók részleteinek a megjelenítéséért. A felhasználók a *ProfileListPage* komponensen keresztül tudnak idenavigálni a kiválasztott felhasználó kártyára kattintva.

- *MyProfileDetailsPage*:
  Ez a komponens felel a bejelentkezett felhasználó részleteinek a megjelenítéséért. A bejelentkezett felhasználók a Profilom menüpontra kattintva tudnak idenavigálni.
- *RegisterModalPage*:
  Ez a komponens felel a regisztrációért. Ez a komponens a többi Page komponenstől eltérően egy modál ablakot (dialog) jelenít meg a Bejelentkezés oldalon. A felhasználók kétféleképpen nyithatják meg ezt a modál ablakot: a menüsávban a Regisztráció menüpontra kattintva: ekkor a felhasználó a Belépés oldalra navigál, majd rögtön megnyílik számára a modál ablak. Másik megnyitási mód, ha a felhasználó a Bejelentkezés oldalon kattint a Regisztrálok gombra.

A Jegyzi webalkalmazásban az alábbi Pure komponenseket hoztam létre:

- *DeleteConfirmModal*:
  Ez a komponens egy megerősítő modál ablak. Amennyiben egy bejelentkezett felhasználó ki szeretné törölni a jegyzetét, ez a komponens megnyílik, hogy a felhasználó megerősítse a szándékát.
- *ModifyRequestEditModal:*
  Ez a komponens felel a Módosítási kérések kezeléséért. A bejelentkezett felhasználók egy modál ablak segítségével tudják kezelni a kiválasztott módosítási kérésüket. Ezt a modált a felhasználók a *ModifyRequestsListPage*-ről tudják elérni.
- *ModifyRequestStatusBadge:*
  Ez a komponens felel a módosítási kérések státuszának a megjelenítéséért.
- *NoteComment:*
  Ez a komponens felel az adott jegyzethez tartozó kommentek megjelenítéséért, új komment létrehozásáért, valamint a bejelentkezett felhasználó által írt kommentek módosításáért, törléséért. Ezt a komponenst a *MyNoteDetailsPage* és a *NoteDetailsPage* tartalmazza.
- *NoteModifyRequest:*
  Ez a komponens felel az adott jegyzethez tartozó új módosítási kérés létrehozásáért. A felhasználók egy modál ablak segítségével tudják létrehozni az új kérést. Ezt a komponenst a felhasználók a *NoteDetailsPage*-ről tudják elérni.

- *NoteReview:*

  Ez a komponens felel az adott jegyzethez tartozó értékelések megjelenítéséért, új értékelés létrehozásáért, valamint a bejelentkezett felhasználó által írt értékelések törléséért. Ezt a komponenst a *MyNoteDetailsPage* és a *NoteDetailsPage* tartalmazza.
- *ProfileModifyModal:*

  Ez a komponens felel a bejelentkezett felhasználó adatainak a módosításáért. A felhasználók egy modál ablak segítségével tudják megadni az új adataikat. Ezt a komponenst a felhasználók a *MyProfileDetailsPage*-ről tudják elérni.
- *Avatar:*

  Ez a komponens felel a felhasználók profilképének a megjelenítéséért, illetve a bejelentkezett felhasználó profil képének a kezeléséért. A komponensre kattintva a felhasználók eljuthatnak az adott felhasználó részletes oldalára.
- *LabelGroup:*

  Ez a komponens felel a jegyzeteknél és szűrőknél megjelenő címkék megjelenítéséért, illetve kiválasztásáért a jegyzet létrehozásánál vagy szűrőfeltétel megadásánál.
- *NoteList:*

  Ez a komponens felel a kilistázott jegyzetek kártyás megjelenítéséért és a jegyzet részletes oldalára való tovább navigálásért. Az összes olyan Page komponens tartalmazza ezt a komponenst, ahol megjelennek kilistázott jegyzetek.
- *ProfileList:*

  Ez a komponens felel a kilistázott felhasználók kártyás megjelenítéséért és a felhasználók részletes oldalára való tovább navigálásért. Az összes olyan Page komponens tartalmazza ezt a komponenst, ahol megjelennek kilistázott felhasználók.
- *NotificationListModal:*

  Ez a komponens felel a bejelentkezett felhasználóhoz érkezett összes értesítés megjelenítéséért és kezeléséért (törlés, illetve státusz módosítás).
- *NotificationDetailsModal:*

  Ez a komponens felel az értesítés részleteinek a megjelenítéséért, illetve a felhasználó itt is tudja az értesítést kezelni.

- *Rating:*
  Ez a komponens felel a jegyzethez érkezett értékelés számának, értékelések átlagának a megjelenítéséért, illetve szűrőfeltétel megadásánál ez a komponens felel az értékelés paraméter megadásáért, és új értékelés létrehozásánál is ezzel a komponens lehet megadni az értékelés számát.
- *SwitchButton:*
  Ez a komponens felel a webalkalmazás témájának a megváltoztatásáért.
- *Title:*
  Ez a komponens felel az oldalak címeinek a megfelelő megjelenítéséért.
- *Header:*
  Ez a komponens felel a webalkalmazás menüsávjának a megfelelő megjelenítéséért.

6.2. Szolgáltatások (Servicek)

A szolgáltatások (service-ek) osztályok, amik egy-egy adott célért felelnek. A webalkalmazásban a legtöbb szolgáltatás a backenddel való kommunikációért felel. Ahhoz, hogy használni tudjuk őket, injektálni kell őket a komponensekben.

- *AuthService:*
  Az authentikációhoz kapcsolódó funkciókat valósítja meg.
- *CommentService:*
  A jegyzetekhez érkezett kommentek lekéréséért és kezeléséért felelős funkciókat valósítja meg.
- *LabelService:*
  A címkék lekérését valósítja meg.
- *ModifyRequestService:*
  A jegyzetekhez érkezett módosítási kérések lekéréséért és kezeléséért felelős funkciókat valósítja meg.
- *NoteService:*
  A jegyzetek lekéréséért és kezeléséért felelős funkciókat valósítja meg.
- *NotificationService:*
  A felhasználónak érkezett értesítések lekéréséét és kezeléséért felelős funkciókat valósítja meg.

- *PaginatorService:*
  Az Angular Material[10] Table lapozásáért felelős funkciókat valósítja meg.
- *ReviewService:*
  A jegyzetekhez érkezett értékelések lekéréséét és kezeléséért felelős funkciókat valósítja meg.
- *ThemeService:*
  A webalkalmazása témájának a kezeléséért felel.
- *ToastService:*
  A webalkalmazásban felugró toast üzenetek létrehozásáért felel.
- *UserService:*
  A felhasználók lekéréséét és kezeléséért felelős funkciókat valósítja meg.
- *TextGenerateService*
  A példafeladatok és válaszok generálásáért felelős funkciókat valósítja meg.
- *FollowedNotesPageService:*
  A *FollowedNotesListPage* komponenshez tartozó szolgáltatás.
- *ModifyRequestPageService:*
  A *ModifyRequestListPage* komponenshez tartozó szolgáltatás.
- *MyNotesPageService:*
  A *MyNotesListPage* komponenshez tartozó szolgáltatás.
- *MyNotesDetailsPageService:*
  A *MyNotesDetailsPage* komponenshez tartozó szolgáltatás.
- *NotePageService:*
  A *NoteDetailsPage* komponenshez tartozó szolgáltatás.
- *MyProfilePageService:*
  A *MyProfilePage* komponenshez tartozó szolgáltatás.
- *ProfilePageService:*
  A *ProfileDetailsPage* komponenshez tartozó szolgáltatás.

6.3. Interfészek (Interface)

Az interfészek segítségével tudjuk megadni, hogy egyes objektumoknak milyen tulajdonságai vannak, és azok milyen típusúak. A webalkalmazásban létrehozott interfészek az adatbázisban tárolt kollekciók alapján kerültek meghatározásra.

- *Comment:*

  A kommentet meghatározó interfész.
- *CommentUpdateRequest:*

  A kommentek módosításáért felelős objektumot meghatározó interfész.
- *Label:*

  A címkéket meghatározó interfész.
- *ModifyRequest:*

  A Módosítási kérést meghatározó interfész.
- *Note:*

  A jegyzetet meghatározó interfész.
- *NoteFilterModel:*

  A jegyzetek listázásáért felelős objektumot meghatározó interfész.
- *Notification:*

  Az értesítést meghatározó interfész.
- *Review:*

  Az értékeléseket meghatározó interfész.
- *User:*

  A felhasználókat meghatározó interfész.
- *UserFilterModel:*

  A felhasználók listázásáért felelős objektumot meghatározó interfész.
- *Education:*

  A felhasználó interfész education objektumának típusát meghatározó interfész.
- *Work:*

  A felhasználó interfész work objektumának típusát meghatározó interfész.
- *Other:*

  A felhasználó interfész other objektumának típusát meghatározó interfész.

6.4. Direktívák (Directive)

A direktívák olyan osztályok, amik további működéseket rendelnek HTML elemekhez. A webalkalmazásban az alábbi direktíva található meg:

- *LabelButton:*
  LabelGroup komponens megjelenését módosítja attól függően, hogy a címke ki lett választva, vagy nem.

6.5. Resolverek

A resolver osztályok olyan adatszolgáltató service-ek, amik navigáció során oldják fel az adatot, így navigáció végén komponens inicializálásakor nem kell várni az adatra. A webalkalmazásban az alábbi resolver-ek találhatóak meg:

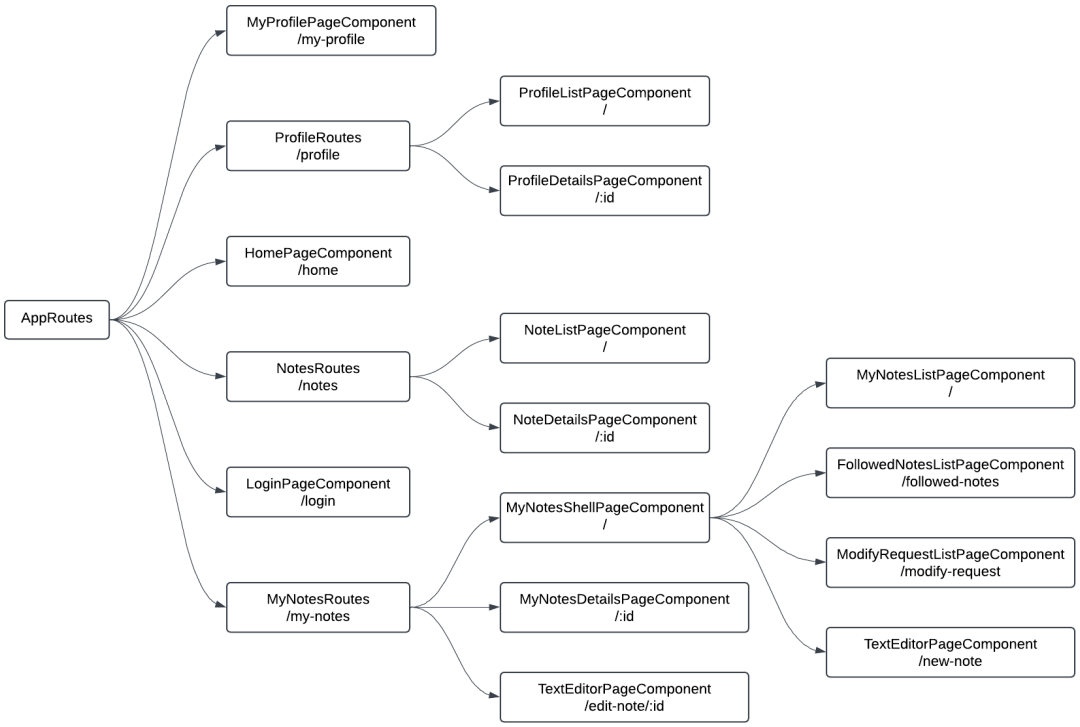
- *MyNote:*
  Az *MyNotesDetailsPage* komponenshez tartozó resolver.
- *Note:*
  Az *NotesDetailsPage* komponenshez tartozó resolver.
- *Profile:*
  A *ProfileDetailsPage, MyProfilePage* komponenshez tartozó resolver.

6.6. Pipeok

A Pipeok lehetővé teszik az adatok formázását a sablonokban (html fájlokban) mielőtt megjelennének. A webalkalmazásban az alábbi pipe-ok találhatóak meg:

- *EducationYear:*
  A felhasználó évfolyamát alakítja át szövegre a felhasználó interfész *education.year* és *eductaion.type* alapján.
- *ElapsedTime:*
  A dátumot alakítja át az adott dátum óta eltelt időre és szöveg formátumra.
- *NoValue:*
  Amennyiben az adott adat null-al vagy undefined-al egyenlő, „Nincs megadva!” szöveg jelenik meg.
- *Type:*
  A profil típusát alakítja át.
- *ToDate:*
  A TimeStamp típusú dátumot alakítja át Date típusú dátumra.

6.7. Routing



6.7. ábra: Routing fa

A felületek közötti navigáció megoldására a webalkalmazás az Angular[1] Routert használja. A Router dolgozza fel az URL-t és alakítja át utasításokká, amik a megjelenő felület cseréjét eredményezik. Az új oldalak nem külön-külön töltődnek be, hanem dinamikusan frissülnek a megjelenő komponensek. Az oldalakat meg tudjuk védeni az illetéktelen hozzáféréstől az Auth Guard segítségével. Az Auth Guard a navigáció előtt ellenőrzi, hogy a felhasználó az oldalra navigálhat-e. Amennyiben nincs ehhez joga a felhasználónak, átirányítja egy másik oldalra. Az Auth Guard segítségével a bejelentkezéshez szükséges oldalakat meg tudtam védeni a vendég felhasználók elől. A Vendég felhasználók elől az alábbi oldalak vannak védve: Jegyzeteim oldalak (*MyNotesRoutes*), Profilom oldal (*MyProfilePageComponent*).

7. Biztonság

A webalkalmazás fejlesztése során kiemelt szempont volt a rendszerhez tartozó adatok, felhasználói műveletek és adminisztrációs funkciók megfelelő védelme. A biztonsági megoldások több rétegben kerültek kialakításra, amelyek magukban foglalják a hitelesítést, a jogosultságkezelést, a kliens- és szerveroldali védelem kialakítását, valamint az adatbevitel ellenőrzését.

7.1. Jogosultsági Model

A rendszer eltérő jogosultsági szintekkel rendelkező felhasználókat kezel. A nyilvános tartalmak – például hírek, eredmények és cikkek – bejelentkezés nélkül is megtekinthetők. Az interaktív funkciók, például kommentelés vagy szavazás kizárólag hitelesített felhasználók számára érhetők el. A tartalomkészítési és adminisztrációs műveletek további szerepkörökhöz kötöttek.

Funkció	Vendég	Felhasználó	Író (Writer)	Szerkesztő (Editor)	Admin
Hírek, Eredmények, Szavazások megtekintése	Igen	Igen	Igen	Igen	Igen
Kommentelés	Nem	Igen	Igen	Igen	Igen
Szavazás	Nem	Igen	Igen	Igen	Igen
Profilkezelés	Nem	Igen	Igen	Igen	Igen
Cikk létrehozása,szerkesztése	Nem	Nem	Igen	Nem	Igen
Eredmények szerkesztése	Nem	Nem	Nem	Igen	Igen

7.2. ábra: Jogosultsági táblázat.

7.3. Backend oldali védelem

Az backend oldal legfőbb szerepe a felhasználók adatainak biztonsága: a felhasználói jelszavak nem olvasható formában, titkosított hash értékként kerülnek tárolásra, Ez megakadályozza, hogy adatbázis-hozzáférés esetén a jelszavak közvetlenül visszafejthetők legyenek.

A szerveroldali alkalmazás ASP.NET Core környezetben készült, ahol a védett végpontok elérését JWT token alapú hitelesítés biztosítja. A sikeres bejelentkezést követően a felhasználó egy időkorlátos tokent kap, amelyet a kliensoldal a későbbi API hívások során használ. A rendszerben alkalmazott JWT token szöveges, Base64URL kódolt formátumú azonosító, amely tipikusan néhány száz karakter hosszúságú. A token három részből áll: fejlécből, adattartalomból és digitális aláírásból.

A tokenek ellenőrzése a backend konfigurációban történik, ahol meghatározásra kerül a titkos kulcs, az aláírási algoritmus és az érvényességi idő. A rendszer kizárólag érvényes és módosítatlan token esetén engedélyezi a hozzáférést.

```
10  # ==========================
11  # JWT Configuration
12  # ==========================
13  JWT_SECRET=your_super_secret_key_here
14  JWT_ISSUER=drs-zona-api
15  JWT_AUDIENCE=drs-zona-client
16  JWT_EXPIRATION_MINUTES=60
```

7.2.2 ábra: Az alkalmazás futtatásához szükséges
környezeti változók sablonfájljának JWT konfigurációjáról szóló része

A jogosultsághoz kötött végpontok elérését az ASP.NET Core által biztosított *Authorize* attribútum védi. Ennek segítségével csak hitelesített felhasználók vagy meghatározott szerepkörrel rendelkező felhasználók férhetnek hozzá bizonyos műveletekhez, amely egyaránt véd külső API hívások esetében is.

```
[Authorize]
[HttpPost( template: "create/{userId:guid}")]
Bence2107
public async Task<ActionResult> Create([FromBody]CommentCreateDto dto, [FromRoute]Guid userId)
{
    var result = await commentService.Create(dto, userId);

    if (!result.IsSuccess)
    {
        return BadRequest( error: new
        {
            result.ErrorField,
            result.Message
        });
    }

    return Ok(result.Value);
}
```

```
[Authorize(Policy = "AuthorOrAdmin")]
[HttpPost( template: "create")]
Bence2107
public async Task<ActionResult> Create([FromBody]ArticleCreateDto dto)
{
    var result = await articleService.Create(dto);

    if (!result.IsSuccess)
    {
        return BadRequest( error: new
        {
            result.ErrorField,
            result.Message
        });
    }

    return Ok(result.Value);
}
```

7.2.3 ábra: Példa az Authorize használatára

7.4. Frontend oldali védelem

A frontend védelme az Angularban használatos útvonalvédelem route guardok segítségével valósul meg. A guardok ellenőrzik, hogy a felhasználó rendelkezik-e érvényes bejelentkezéssel, illetve megfelelő jogosultsági szinttel. Ha igen, átengedi, ha nem, a megfelelő útvonalra visszadobja, megakadályozva a kérést.

```
export const authGuard: CanActivateFn = () : UrlTree | Observable<true | UrlTree> => {  Show usages  Bence2107
  const authService : AuthService = inject(AuthService);
  const router : Router = inject(Router);

  const me$ : Observable<UserProfileResponse> | null = authService.getMe();

  if (!me$) return router.createUrlTree(['/auth']);

  return me$.pipe(
    map(user : UserProfileResponse => user ? true : router.createUrlTree(['/auth'])),
    catchError(() : Observable<UrlTree> => of(router.createUrlTree(['/auth'])))
  );
};

export const guestGuard: CanActivateFn = () : true | Observable<boolean | UrlTree> => {  Show usages  Bence2107
  const authService : AuthService = inject(AuthService);
  const router : Router = inject(Router);

  const me$ : Observable<UserProfileResponse> | null = authService.getMe();

  if (!me$) return true;

  return me$.pipe(
    map(user : UserProfileResponse => user ? router.createUrlTree(['/profile', authService.currentUser()?.username]) : true),
    catchError(() : Observable<boolean> => of( value: true))
  );
};
```

7.2.4 ábra: A Bejelentkezés útvonal védelmének implementációja

A frontend oldalon HTTP interceptor került alkalmazásra, amely automatikusan hozzáfűzi a JWT tokent a backend felé küldött API kérésekhez. Ez biztosítja, hogy a védett végpontokhoz tartozó hitelesítés egységesen és automatikusan történjen.

```
export const authInterceptor: HttpInterceptorFn  Show usages  Bence2107 *
  = (req : HttpRequest<unknown> , next : HttpHandlerFn ) : Observable<HttpEvent<unknown>> => {
  const token : string | null = inject(AuthService).getToken();

  if (token) {
    req = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });
  }

  return next(req);
};
```

7.2.5 ábra: Az autentikációt ellenőrző HTTP interceptor implementációja

7.5. Adatvalidáció

A hibás vagy hiányos adatbevitel megelőzése érdekében a rendszer kliens- és szerveroldali validációt egyaránt alkalmaz.

Frontend oldalon Angular Reactive Forms segítségével történik az űrlapok ellenőrzése. A rendszer figyeli a kötelező mezőket, az e-mail formátumot, a minimális karakterszámot, valamint a jelszavak egyezőségét.

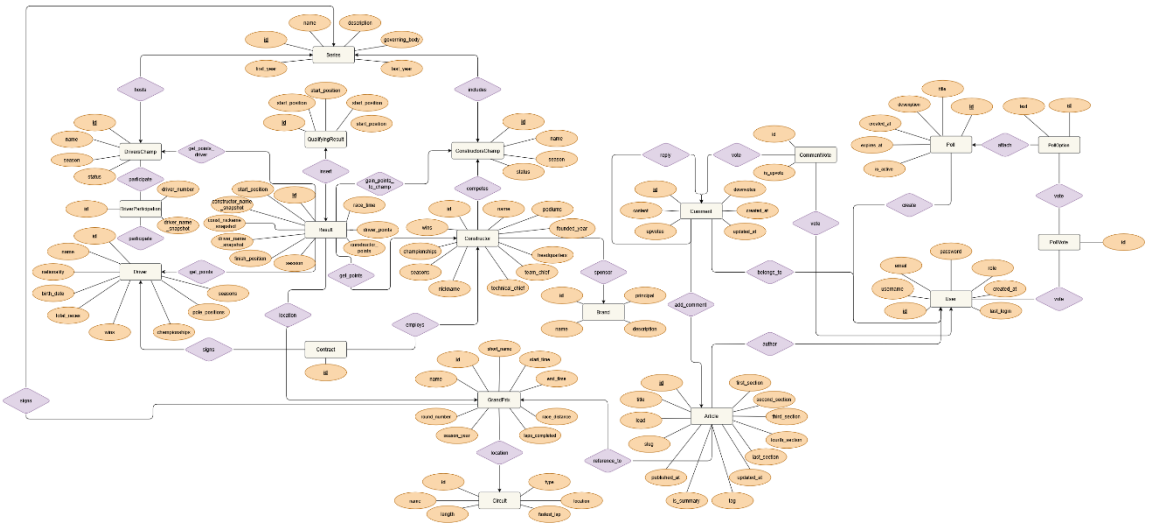
Backend oldalon a beérkező adatok ellenőrzése DTO (Data to Object) modelleken elhelyezett validációs szabályokkal történik. Ennek köszönhetően a hibás adatok akkor sem kerülhetnek feldolgozásra, ha a kliensoldali ellenőrzés megkerülésre kerül.

7.6. Összegzés

A kialakított biztonsági megoldások több rétegben védik a rendszert. A hitelesítés JWT tokenekkel történik, a jogosultságkezelést szerepkörök szabályozzák, az útvonalak és végpontok védettek, míg a validációs mechanizmusok biztosítják a bevitt adatok helyességét. Ennek eredményeként a rendszer alkalmas valós felhasználói környezetben történő biztonságos működésre.

8. Adatmodell

A rendszer adattárolása relációs adatbázisban valósul meg PostgreSQL használatával. Az adatmodell több logikai alrendszerre bontható: felhasználókezelés, hírtartalom-kezelés, szavazási modul, valamint motorsport eredménykezelési modul. Az adatbázis szerkezete normalizált formában került kialakításra, amely csökkenti az adatredundanciát és biztosítja az adatok konzisztenciáját.



8.1. ábra: A Webalkalmazáshoz készült egyed kapcsolat diagram

8.1. Felhasználói Modul

8.1.1 Felhasználó (User)

A User entitás tárolja a rendszer felhasználóinak adatait. Ide tartozik többek között a felhasználónév, teljes név, e-mail cím, titkosított jelszó, szerepkör, valamint a bejelentkezési aktivitással kapcsolatos információk. A rendszer jogosultságkezelése erre az entitásra épül.

8.2 Hír Modul

8.2.1 Hír (Article)

Az Article entitás a hírcikkek és összefoglalók tárolására szolgál. Tartalmazza a cikk címét, bevezető szövegét, több tartalmi szekcióját, URL azonosítóját (slug), kategóriáját, szerzőjét és publikálási időpontját.

8.2.2 Komment (Comment)

A Comment entitás a cikkekhez kapcsolódó hozzászólásokat kezeli. Egy komment kapcsolódik egy felhasználóhoz és egy cikkhez, valamint támogatja a válaszkommenteket önhivatkozó kapcsolat segítségével.

8.2.3 Komment szavazás (CommentVote)

A CommentVote entitás a hozzászólások értékelését kezeli. Segítségével a felhasználók pozitív vagy negatív szavazatot adhatnak egy kommentre.

8.3 Szavazási Modul

8.3.1 Szavazás (Poll)

A Poll entitás a rendszerben létrehozott szavazásokat reprezentálja. Tartalmazza a kérdést, leírást, kategóriát, készítő felhasználót, lejárati időt és az aktív állapotot.

8.3.2 Szavazás opció (PollOption)

A PollOption entitás egy adott szavazás választlehetőségeit tárolja. Egy szavazáshoz több válaszopció is tartozhat.

8.3.3 Szavazat (Szavazásra) (PollVote)

A PollVote entitás rögzíti, hogy egy felhasználó melyik választlehetőségre szavazott. Ez biztosítja az egyedi és visszakövethető szavazást.

8.4 Versenypálya Modul

8.4.1 Versenypálya (Circuit)

A Circuit entitás a versenypályák adatait kezeli. Tárolja a pálya nevét, hosszát, típusát, földrajzi helyét és leggyorsabb köradatait.

8.4.2 Nagydíj (GrandPrix)

A GrandPrix entitás egy adott futam eseményt reprezentál. Kapcsolódik egy versenypályához és sorozathoz, valamint tartalmazza a szezonévet, fordulószámot, időpontot és futamtávot.

8.5 Eredmények Modul

8.5.1 Márka (Brand)

A Brand entitás az autógyártó márkákat reprezentálja. Az adatmodell elkülöníti a márkát és a versenycsapatot, így több csapat is kapcsolódhat ugyanazon márkához.

8.5.2 Konstruktor (Constructor)

A GrandPrix entitás egy adott futam eseményt reprezentál. Kapcsolódik egy versenypályához és sorozathoz, valamint tartalmazza a szezonévet, fordulószámot, időpontot és futamtávot.

8.5.3 Konstruktor nevezés (ConstructorCompitition)

A ConstructorCompetition kapcsolótábla rögzíti, hogy mely csapatok indulnak az adott konstruktőri bajnokságban.

8.5.4 Konstruktori bajnokság (ConstructorsChampionship)

A ConstructorsChampionship entitás a csapatbajnokság szezonális adatait tárolja.

8.5.5 Szerződés (Contract)

A Contract entitás a versenyzők és csapatok közötti szerződéses kapcsolatot modellezi, amely lehetővé teszi a pilóták aktuális csapatához történő automatikus hozzárendelést.

8.5.6 Pilóta (Driver)

A Driver entitás a versenyzők törzsadatait tartalmazza. Ide tartozik a név, nemzetiség, születési dátum, valamint statisztikai adatok, például győzelmek és világbajnoki címek száma.

8.5.7 Pilóta részvétel (DriverParticipation)

A DriverParticipation kapcsolótábla kezeli, hogy mely versenyzők vesznek részt egy adott bajnokságban. Tartalmaz rajtszámot és pillanatképként tárolt nevet is.

8.5.8 Egyéni bajnokság (DriversChampionship)

A DriversChampionship entitás az adott szezon versenyzői bajnokságát írja le. Egy sorozathoz több szezon is tartozhat.

8.5.9 Időmérő eredmény (QualifyingResult)

A QualifyingResult entitás az időmérő edzés eredményeit kezeli, külön tárolva a Q1, Q2 és Q3 szakaszok idejét.

8.5.10 Eredmény (Result)

A Result entitás a futamok eredményeit tárolja. Tartalmazza a rajtpozíciót, célpozíciót, pontokat, körszámot, státuszt, időeredményt, valamint a kapcsolódó pilóta és csapat adatait.

8.5.11 Széria (Series)

A Series entitás különböző versenysorozatok adatait tárolja, például Formula–1 vagy Formula–2. Tartalmazza a sorozat nevét, rövidítését, leírását, irányító szervezetét és pontozási rendszerét.

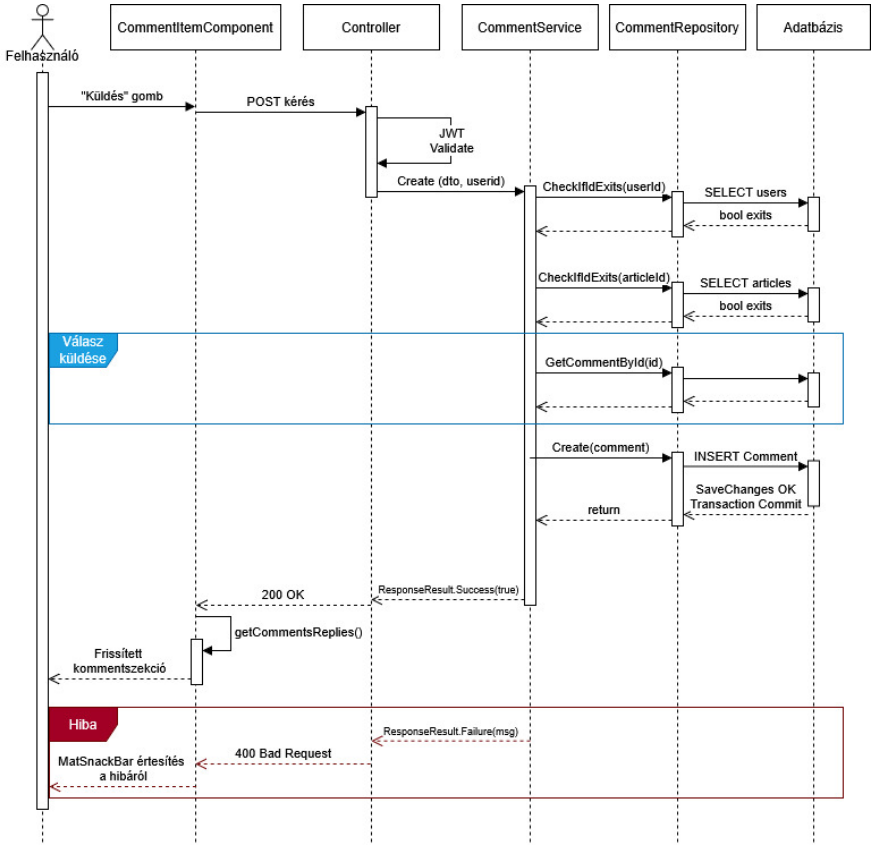
8.6 Kapcsolatok

Az adatmodell három fő logikai egységre bontható: felhasználói modul, tartalomkezelő modul és motorsport eredménykezelő modul. A felhasználói modul kezeli a regisztrációt, jogosultságokat, hozzászólásokat és szavazásokat. A tartalomkezelő rész a cikkek, kategóriák és közösségi interakciók kezelését biztosítja. A motorsport modul a versenysorozatok, futamok, pilóták, csapatok és eredmények nyilvántartására szolgál. Ennek befutó (mindent összekötő) kapcsolata az Eredmény (Result) entitás: ide fut be a pilóta, a csapat, az egyéni és konstruktőri bajnokság, és a nagydíj is.

A kapcsolatok többsége egy–több típusú, míg a több–több kapcsolatokat kapcsolótáblák valósítják meg.

9. A rendszer magasszintű folyamatai

A webalkalmazás magasszintű folyamatai közé tartozik a Kommentelés és kommentre válasz írása.



9.1. ábra: A komment létrehozásának szekvencia diagramja

A 9.1-es ábra a kommentre adott válasz (reply) létrehozásának teljes folyamatát mutatja be a kliensoldali Angular alkalmazástól egészen az adatbázisig. A folyamat a submitReply() metódus meghívásával indul, amikor a felhasználó elküldi a válaszát. Az Angular komponens összeállítja a szükséges CommentCreateDto objektumot, majd a CommentService.createComment() segítségével egy POST */api/Comments/create/{userId}* HTTP kérést küld a backend felé.

A kérés beérkezése után az ASP.NET Core rendszer először hitelesítési ellenőrzést végez az [Authorize] attribútum alapján. A middleware validálja a Bearer tokent, és csak érvényes token esetén engedi tovább a kérést a controllerhez. Ha a token hibás vagy lejárt, a folyamat megszakad, és a rendszer hibaüzenettelválaszt küld vissza.

Sikeres hitelesítés után a controller fogadja a kérést, de üzleti logikát nem tartalmaz, csupán továbbítja azt a service réteg felé a commentService.Create(dto, userId) hívással. A tényleges feldolgozás itt történik: a service ellenőrzi, hogy a felhasználó létezik-e, a megadott cikk létezik-e, valamint, ha válaszkomment készül, akkor a szülőkomment is megtalálható-e. Ezek külön adatbázis-lekérdezésekkel történnek.

Ha minden validáció sikeres, a service létrehozza az új Comment entitást, majd meghívja a commentsRepo.Create() metódust. Ez Entity Framework Core használatával végrehajtja az AddAsync() és SaveChangesAsync() műveleteket, amelyek tranzakciós módon mentik az adatot a PostgreSQL adatbázisba.

Sikeres mentés esetén a backend *ResponseResult<bool>.Success(true)* választ ad vissza, amely 200 OK HTTP státusszal jut vissza az Angular alkalmazáshoz. A frontend ezt a next callbackben kezeli: újratölti a válaszkommenteket a getCommentsReplies() hívással, frissíti a megjelenített válaszlistát, és láthatóvá teszi azt a felhasználó számára.

Hiba esetén – például sikertelen validációkor – a service ResponseResult.Failure() eredményt küld vissza, amelyből a controller 400 BadRequest választ készít. Az Angular ezt érzékeli, és a replyError változóban jeleníti meg a megfelelő hibaüzenetet. A diagram tehát jól szemlélteti a kliens–szerver kommunikáció, a hitelesítés, a rétegezett architektúra és a hibakezelés teljes működését egy kommentválasz létrehozásán keresztül.

10. Fontosabb kódrészletek

10.1. Regisztráció

```
signup(): void {  Show usages  Bence2107
  if (this.registerForm.invalid) return;

  const request: RegisterRequest = {
    email: this.regEmail.value,
    fullName: this.regFullName.value,
    username: this.regUsername.value,
    password: this.regPassword.value,
  };

  this.authService.register(request).subscribe({
    next: () :void  => {
      this.router.navigate(['/home']);
      this.snackBar.openFromComponent(CustomSnackbarComponent, {
        data: { message: 'Sikeres Regisztráció', actionLabel: 'Rendben' },
        duration: 3000,
        horizontalPosition: 'center',
      });
    },
    error: (err: HttpValidationError) :void  => this.applyServerErrors(this.registerForm, err)
  });
}
```

10.1.1. ábra: Az AuthComponent signup metódusa
(A regisztráció frontend oldali logikája)

A komponens inicializálásakor először ellenőrzi, hogy a szerver elérhető-e a checkConnection() metódus segítségével. Amíg a kapcsolat ellenőrzése zajlik, egy töltőcsíkot jelenít meg a felhasználónak, ha pedig a szerver nem elérhető, egy hibaüzenetet mutat újrapróbálkozási lehetőséggel.

A regisztrációs űrlap Angular Reactive Forms segítségével épül fel, ahol a mezőkre kliens oldali validációk vannak definiálva, például kötelező kitöltés, email formátum ellenőrzés, maximális karakterszám. Az űrlap beküldésekor a signup() metódus hívódik meg, amely összeállítja a RegisterRequest objektumot és átadja az AuthService-nek.

Egy különösen érdekes megoldás a szerver oldali hibák kezelése, amelyet az applyServerErrors() metódus valósít meg. Ez a metódus a backend által visszaküldött validációs hibákat mezőnként visszamappolja az Angular form controlokra, így a hibaüzenetek közvetlenül a megfelelő mező alatt jelennek meg. Emellett a clearServerErrorOnChange() gondoskodik arról, hogy ha a felhasználó módosítja a mezőt, a szerver oldali hibaüzenet automatikusan eltűnjön.

```
4+5 usages   Bence2107
public async Task<ResponseResult<AuthResponse>> Register(RegisterRequest request)
{
    if (await authRepository.UserExistsByUsername(request.Username))
    {
        return ResponseResult<AuthResponse>.Failure(
            field: "username",
            message: "Ez a felhasználónév foglalt."
        );
    }

    if (await authRepository.UserExistsByEmail(request.Email))
    {
        return ResponseResult<AuthResponse>.Failure(
            field: "email",
            message: "Ez az Email Cím már használatban van."
        );
    }
```

```
        try
        {
            var user = new User
            {
                Id = Guid.NewGuid(),
                Username = request.Username,
                FullName = request.FullName,
                Email = request.Email,
                PasswordHash = BCrypt.Net.BCrypt.HashPassword(request.Password),
                Role = "User",
                Created = DateTime.UtcNow,
                HasAvatar = false,
                IsLoggedIn = false
            };

            await authRepository.Create(user);
            var authResponse = await GenerateAuthResponse(user);

            return ResponseResult<AuthResponse>.Success(authResponse);
        }
        catch (Exception ex)
        {
            return ResponseResult<AuthResponse>.Failure(
                message: $"Registration failed: {ex.Message}"
            );
        }
    }
```

10.1.2. ábra: A felhasználó létrehozásának backend oldali logikájának szerviz szintű implementációja

A Register() metódus először ellenőrzi, hogy a megadott felhasználónév és email cím még nem foglalt-e, és ha igen, mezőspecifikus hibaüzenettel tér vissza a ResponseResult pattern segítségével. Ez egy egyedi megoldás, ami egységes hibakezelést biztosít az egész alkalmazásban.

Ha az ellenőrzések sikeresek, a jelszó BCrypt algoritmussal hashelve kerül eltárolásra, soha nem nyers szövegként. Ezután a GenerateAuthResponse() metódus hívódik meg, amely egy egyedi session azonosítót generál, majd a GenerateJwtToken() segítségével elkészíti a JWT tokent. A token tartalmazza a felhasználó azonosítóját, email címét, nevét, szerepkörét és a session azonosítót claim-ekként, és HMAC-SHA256 algoritmussal kerül aláírásra. Sikeres regisztráció után a felhasználó azonnal be is van jelentkezve, és a token visszakerül a frontendre.

```
register(request: RegisterRequest): Observable<AuthResponse> {  Show usages  Bence2107
  return apiAuthRegisterPost$Json(this.http, this.apiConfig.rootUrl, {body: request}).pipe(
    map(response :HttpResponse<AuthResponse> & {readonly b...  => response.body),
    tap(response :AuthResponse  => {
      localStorage.setItem(this.TOKEN_KEY, JSON.stringify(response));
      this.currentUser.set(response);
      this.loadProfile();
    })
  )
}
```

10.1.3. ábra: A felhasználó létrehozásának szerviz szintű backend oldali logikája

Ha a backend 200-as válasszal tér vissza, a frontend átirányít a főoldalra, és frissíti az aktuális felhasználót, és a „Sikeres Regisztráció” üzenettel az oldal alján visszatér.

10.2. Az Autentikáció kezelése a frontenden

Az frontenden leimplementált AuthService osztály felelős a teljes autentikációs folyamat kezeléséért a frontend oldalon. A szolgáltatás Angular signal segítségével tartja számon a bejelentkezett felhasználó adatait, ami lehetővé teszi, hogy az alkalmazás bármely komponense reaktívan reagáljon a bejelentkezési állapot változásaira.

```
register(request: RegisterRequest): Observable<AuthResponse> {  Show usages  Bence2107
  return apiAuthRegisterPost$Json(this.http, this.apiConfig.rootUrl, {body: request}).pipe(
    map(response :HttpResponse<AuthResponse> & {readonly b...  => response.body),
    tap(response :AuthResponse  => {
      localStorage.setItem(this.TOKEN_KEY, JSON.stringify(response));
      this.currentUser.set(response);
      this.loadProfile();
    })
  )
}

login(request: LoginRequest): Observable<AuthResponse> {  Show usages  Bence2107
  return apiAuthLoginPost$Json(this.http, this.apiConfig.rootUrl, { body: request }).pipe(
    map(response :HttpResponse<AuthResponse> & {readonly b...  => response.body),
    tap(response :AuthResponse  => {
      localStorage.setItem(this.TOKEN_KEY, JSON.stringify(response));
      this.currentUser.set(response);
      this.loadProfile();
    })
  );
}
```

10.2.1. ábra: Az AuthService komponensben leimplementált *register* és *login* metódusok forráskódja

Bejelentkezés és regisztráció után a szerver által visszaküldött JWT token és a hozzá tartozó adatok localStorage-ban kerülnek eltárolásra. Az alkalmazás indulásakor a loadFromStorage() metódus gondoskodik arról, hogy ha a felhasználónak van érvényes, el nem tárolt munkamenete, automatikusan visszaálljon a bejelentkezett állapot anélkül, hogy újra be kellene jelentkeznie.

```
isLoggedIn(): boolean {  Show usages  Bence2107
  const user :AuthResponse | null  = this.currentUser();
  if (!user?.token || !user?.expiresAt) return false;
  return new Date(user.expiresAt) > new Date();
}
```

10.2.2. ábra: Az AuthService komponensben leimplementált *isLoggedIn* metódus forráskódja

Az isLoggedIn() metódus nem csupán azt ellenőrzi, hogy létezik-e token, hanem a token lejárati idejét is megvizsgálja. Ha a token lejárt, a felhasználó automatikusan kijelentkezettnek számít még akkor is, ha a localStorage-ban szerepel a token.

```
export class AuthService {
  private readonly TOKEN_KEY :"auth_token"  = 'auth_token';

  currentUser :WritableSignal<AuthResponse | null>  = signal<AuthResponse | null>(this.loadFromStorage());
  currentProfile :WritableSignal<UserProfileResponse | null...  = signal<UserProfileResponse | null>( initialValue: null);

  constructor(private http: HttpClient, private router: Router, private apiConfig: ApiConfiguration) {}
```

10.2.3. ábra: Az AuthService komponensben leimplementált *signal-ok* forráskódja

A currentProfile signal tárolja a felhasználó részletes profiladatait, amelyek bejelentkezés, regisztráció és profilmódosítás után automatikusan frissülnek a loadProfile() metódus hívásával. Ez biztosítja, hogy az oldalon mindig naprakész adatok jelenjenek meg.

```
logout(): void {  Show usages  Bence2107
  apiAuthLogoutPost(this.http, this.apiConfig.rootUrl).subscribe({
    complete: () :void  => {
      this.clearSession();
    }
  });
}
```

```
private clearSession(): void {  Show usages  Bence2107
  localStorage.removeItem(this.TOKEN_KEY);
  this.currentUser.set( value: null);
  this.currentProfile.set( value: null);
  this.router.navigate(['/home']);
}
```

10.2.4. ábra: Az AuthService komponensben leimplementált *logout* metódus forráskódja

A clearSession() metódus törli a localStorage-ból a tokent, nullra állítja a signalokat, és visszanavigál a főoldalra, így teljesen tiszta állapotot hagy maga után.

10.3. Nagydíj események lekérése

```
0+4 usages   Bence2107
public async Task<List<string>> GetAvailableSessionsByGrandPrixId(Guid grandPrixId)
{
    return await _results // DbSet<Result>
        .AsNoTracking()
        .Where(r :Result => r.GrandPrixId == grandPrixId) // IQueryable<Result>
        .Select(r :Result => r.Session)
        .Distinct() // IQueryable<string>
        .OrderBy(s :string => s) // IOrderedQueryable<string>
        .ToListAsync(); // Task<List<...>>
}
```

10.2.1 ábra: A nagydíj események lekéréseire szolgáló logika Repository rétege

A GetAvailableSessionsByGrandPrixId() metódus EF Core segítségével lekéri az adott Grand Prix-hez tartozó összes egyedi session típust. Az AsNoTracking() hívás teljesítményoptimalizálási célt szolgál, mivel csak olvasási műveletről van szó, így nincs szükség a változáskövetésre. A Distinct() gondoskodik arról, hogy minden session típus csak egyszer szerepeljen az eredményben, az OrderBy() pedig ábécé sorrendbe rendezi őket.

```
loadSessions() :void { Show usages   Bence2107
  this.isLoading.set( value: true);
  this.standingsService.getSessionsByGrandPrix(this.gpId()).subscribe({
    next: sessions :string[] => {
      this.sessions.set(sessions);
      if (sessions.length > 0) {
        this.onSessionChange(sessions[0]);
      } else {
        this.isLoading.set( value: false);
      }
    },
    error: () :void => this.isLoading.set( value: false)
  });
}
```

10.2.2 ábra: A nagydíj események lekéréseire szolgáló logika Repository rétege

A loadSessions() metódusban látható ahogy a frontend service-t felhasználja a komponens. Betöltés közben az isLoading signal true értékre van állítva, és ha vannak sessionök, automatikusan az első session kerül kiválasztásra az onSessionChange() meghívásával.

11. A Mesterséges intelligencia használata a fejlesztés során

A szakdolgozat elkészítése során a fejlesztési folyamat több szakaszában alkalmaztam mesterséges intelligencián alapuló eszközöket. Ezek elsősorban a programozási feladatok gyorsítását, az ismétlődő munkafolyamatok támogatását, valamint a hibakeresés hatékonyságának növelését szolgálták. A fejezet célja annak bemutatása, hogy milyen eszközöket használtam, milyen területeken bizonyultak hasznosnak, illetve hogyan történt a kapott eredmények ellenőrzése.

11.1 Alkalmazott eszközök és szerepük

A fejlesztés során két mesterséges intelligencia alapú rendszert használtam: a Claude és a Google Gemini megoldásait. A használatuk során fokozatosan kialakult, hogy mely feladatokra melyik eszköz bizonyult hatékonyabbnak.

A Claude rendszert elsősorban kódgenerálásra és tesztírásra alkalmaztam. Segítségével gyorsan készíthetők voltak webes komponensek, segédfüggvények, valamint ismétlődő szerkezetű kódrészletek. Emellett egységtesztek létrehozásában is hasznosnak bizonyult, mivel meglévő függvények alapján rövid idő alatt tudott kiindulási alapot biztosítani.

A Google Gemini rendszert inkább hibakeresésre, hibajavítási ötletek kérésére, valamint felhasználói felülettel kapcsolatos kérdésekben alkalmaztam. Egyes hibák feltárásában és vizuális kialakítási döntésekben gyakorlati segítséget nyújtott.

11.2 Konkrét felhasználási területek

A leggyakoribb alkalmazási terület a kódgenerálás volt. Ide tartoztak például API-kezeléssel kapcsolatos logikai elemek, egyszerűbb komponensek, valamint boilerplate jellegű kódrészletek elkészítése. Ezeknél a feladatoknál az MI-eszközök jelentős időmegtakarítást eredményeztek.

Szintén rendszeresen használtam mesterséges intelligenciát tesztírásra. A generált teszteket minden esetben átnéztem, szükség esetén módosítottam, illetve kiegészítettem további esetekkel. Ez különösen hasznosnak bizonyult, mivel a tesztelés fontos, ugyanakkor időigényes része a fejlesztési folyamatnak.

Hibajavítás során elsősorban a Gemini rendszert alkalmaztam. Hibaüzenetek és problémás kódrészletek alapján több esetben adott olyan javaslatokat, amelyek segítették a hiba gyorsabb feltárását. A kapott válaszokat azonban közvetlenül nem használtam fel.

11.3 Validálás és kritikai ellenőrzés

A mesterséges intelligencia által generált eredményeket minden esetben ellenőriztem. A kapott megoldásokat nem kész válaszként, hanem javaslatként kezeltem. Több alkalommal előfordult, hogy a generált kód működőképesnek tűnt, azonban futtatás során hibát okozott, vagy nem illeszkedett megfelelően a rendszer többi részéhez.

Az ellenőrzés során manuális tesztelést, a kód logikai áttekintését, dokumentációval való összevetést, valamint a meglévő automatikus tesztek futtatását alkalmaztam. Ezek alapján megállapítható, hogy az MI-eszközök használata mellett továbbra is szükséges a fejlesztői kontroll és szakmai ellenőrzés.

11.4 Területek, ahol nem alkalmaztam MI-t

Bizonyos részeket tudatosan mesterséges intelligencia használata nélkül készítettem el. A rendszer központi üzleti logikáját, valamint a fontosabb algoritmusokat saját tervezés alapján valósítottam meg. Ezek a részek mélyebb megértést igényeltek, ezért célszerűbbnek tartottam önállóan kidolgozni őket.

A szakdolgozat elemző és értékelő részei szintén saját gondolatokra és tapasztalatokra épültek. Ezek elkészítését nem bíztam automatizált eszközökre.

11.5 Összegzés

A mesterséges intelligencia alapú eszközök alkalmazása jelentősen támogatta a fejlesztési munkát, különösen az ismétlődő feladatok, egyszerűbb kódrészletek és tesztek elkészítése során. Ugyanakkor a kapott eredmények ellenőrzése minden esetben elengedhetetlen volt.

Összességében az MI-eszközöket hatékony támogató eszköznek tekintem, amelyek megfelelő használat mellett növelik a fejlesztés hatékonyságát, azonban nem helyettesítik a szakmai döntéshozatalt és a fejlesztői felelősséget.

12. Tapasztalatok, továbbfejlesztési lehetőségek

A szakdolgozatom készítése előtt már tisztában voltam azzal, hogy Angular -t fogok használni frontendre. Ennek az oka az volt, hogy az Angular keretrendszer használatával már van tapasztalatom, szeretek vele dolgozni, emellett az itthoni piacon eléggé elterjedt.

A Backend kiválasztásában sok szerepet játszott a képek kezelése, amit egy egyszerűbb backend, mint pl. a Firebase nem tudta volna kielégíteni. Mivel korábban sokat Java-ztam, azonban a Spring Boot használatát erősen mellőznöm kellett. Egy szaktársam, valamint egy kurzus vitt el a C# (ASP.NET és Entity Framework Core) irányába, ami a korábban említett programozási nyelv egy egyszerűsített változata, meghagyva az objektum orientált gondolkodást. A projekt tervezése előtt készítettem magamnak egy működő alap appot a később választott technológiával és struktúrával a gyorsabb implementáció érdekében.

Ezután elkezdtem a projekt adatbázissémájának megvalósítását: létrehoztam egy akkor még jónak gondolt EK (Egyed-Kapcsolat) diagrammot. A rendszer tervezése során ugyanis éreztem, hogy amit csinálok az kevés lesz: ekkor lett megtervezve a pontszámító rendszer, ami az eredményeket nem kívülről, hanem az adatbázisból.

Mivel törekedtem a Backend-First megvalósításra, a backenddel kezdtem: létrehoztam az alap struktúrát, majd létrehoztam a GitHub Repository-t is. Utóbbival már volt tapasztalatom, így a különböző branch (ág) stratégiákat is ismertem: A funkción alapuló ágak különböző Issue-k (problémák) alapján jöttek létre: pl. dokumentáció, adatbázisséma leimplementálása backend oldalon stb. A leghosszabb időt a különböző backend oldali Repository-k írása volt. Át kellett gondolnom felületi tervek nélkül a rendszer igényeit, így voltak implementációk, interfészek, amik később törlésre kerültek.

Megcsúszva a határidővel, hozzá kellett kezdjek a frontend oldalhoz: elsősorban a hírekre, híroldalra és a kommentelés funkcióira koncentráltam. Ez volt a projekt első éles tesztelése is egyben: létrehoztam az alap kinézetet, amit egy korábbi, ehhez nagyon hasonló projektemből alakítottam át. Ebből tanulva álltam neki a struktúra átgondolásának, a backend logika teljes felszámolásának, egy teljesen más irányt vett a projekt

A frontend oldalak elkészítése több hetet vett igénybe: minden UI/UX elvre igyekeztem figyelni, emellett az oldal backend logikáját folyamatosan igazítottam a

meglévő frontendhez. Ezt a részét eleinte nagyon élveztem, azonban hamar rájöttem, hogy a projekt igényei messze nem egy emberre tervezett. Sok oldallal és megoldással a tervezett megoldási idő felett lett megvalósítva. Mivel rájöttem mindennel nem tudok egyszerre, a projekt fejlesztésének két harmadánál segítséget hívtam: Az eredményeket a párom vezette folyamatosan egy külön táblázatban, hogy minden eredményt be tudjak írni előre, emellett

Azonban ki kell emelnem, hogy a projekt kódolását mindig is élveztem: élveztem az oldalak igazgatását, a backend logika folyamatos kiegészítését és hiányosságainak bepótlását.

Sok olyan funkció van, amit nem bírtam megvalósítani időhiány, és rendszertervezési hibái miatt. A legfontosabb ezek közül az eredmény rendszerem hiányosságai: mivel minden sportág máshogy számolja a bajnokságait, illetve eleve más bajnokságok is vannak, ehhez egy külön rendszer kellene több adatbázissal, más struktúrával. A jelenlegi rendszer csak az F1 és F2 pontozási rendszerét elégíti ki, azt viszont automatizáltan, felhasználó baráti módon. Emellett néhol a loading state képernyők nem egységesek, illetve egy mobil alkalmazás még inkább feldobná a projektet: ehhez azonban új rendszer kell, más API hívások és logika.

Összességében úgy gondolom, hogy egy nagyon komplex webalkalmazást sikerült létrehoznom különböző kisebb alkalmazások összevonásával: egy közösségi oldal jellegű hír oldalt, ahol az újságírók és a rajongók egymással vitatkozhatnak kommentek formájában, egy szavazási felület, egy eredményt listázó és managelő rendszert, egy profilt kezelő felület egy nagy alkalmazásban. A projekt készítése során emellett sokat tanultam magamról és a szakmámról. Előbbivel kapcsolatban bebizonyítottam, hogy a sok kitartással a lehetetlennek gondolt dolgokat is meg lehet oldani. Utóbbival kapcsolatban pedig, hogy egy szoftver sohasem lesz tökéletes: mindig van valami, amin javítani. foltozni kell.

Irodalomjegyzék (nem frissített)

1. Angular: https://angular.dev/ (Utolsó megtekintés: 2024. 11. 14.)
2. Google Firebase: https://firebase.google.com/ (Utolsó megtekintés: 2024. 12. 14.)
3. Visual Studio Code: https://code.visualstudio.com/ (Utolsó megtekintés: 2024. 12. 14.)
4. Prezi: https://prezi.com/hu/ (Utolsó megtekintés: 2024. 11. 10.)
5. Padlet: https://padlet.com/ (Utolsó megtekintés: 2024. 11. 10.)
6. Wakelet: https://wakelet.com/ (Utolsó megtekintés: 2024. 11. 10.)
7. GoConqr: https://www.goconqr.com/ (Utolsó megtekintés: 2024. 11. 10.)
8. Figma: https://www.figma.com/ (Utolsó megtekintés: 2024. 12. 14.)
9. Tailwind: https://tailwindcss.com/ (Utolsó megtekintés: 2024. 11. 14.)
10. Angular Material: https://material.angular.io/ (Utolsó megtekintés: 2024. 11. 14.)
11. Tailwind Animated: https://www.tailwindcss-animated.com/ (Utolsó megtekintés: 2024. 11. 14.)
12. Quill: https://quilljs.com/ (Utolsó megtekintés: 2024. 11. 14.)
13. Phosphor Icons: https://phosphoricons.com/ (Utolsó megtekintés: 2024. 11. 14.)
14. Toaster: https://github.com/scttcper/ngx-toastr (Scott Cooper) (Utolsó megtekintés: 2024. 11. 14.)
15. Skeleton Loader: https://github.com/willmendesneto/ngx-skeleton-loader (Will Mendes) (Utolsó megtekintés: 2024. 11. 14.)
16. Angular Fire: https://firebaseopensource.com/projects/firebase/angularfire/ (Utolsó megtekintés: 2024. 11. 14.)
17. Gemini Ai: https://ai.google.dev/ (Utolsó megtekintés: 2024. 11. 20.)
18. Github: https:/github.com/ (Utolsó megtekintés: 2024. 12. 14.)

Nyilatkozat

Alulírott Szabó Bence programtervező informatikus szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztés Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök stb.) használtam fel. Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumában tárolja.

Dátum: 2026.05.03.

Aláírás

Köszönetnyilvánítás

Szeretnék köszöntet mondani elsősorban a témavezetőmnek, Dr. Bilicki Vilmos egyetemi adjunktusnak, amiért segítette és felügyelte a munkámat.

Emellett szeretnék köszönetet mondani a családomnak, akik teljes szívükkel támogattak nemcsak a szakdolgozatomban, hanem az egyetemi, és az oda vezető tanulmányi éveimet is.

Továbbá szeretnék köszönetet mondani két barátomnak, akik segítettek a projekt kitelepítésében, fejlesztésében és szakmai meglátásaikkal egyenesbe hozták a projekt struktúráját.

Végül, de nem utolsó sorban a páromnak is köszönöm, hogy a projektemet megtöltötte tartalommal és a kezdetek óta szívügyeként kezelte. ezzel is támogatva engem és a tanulmányaimat.

Elektronikus mellékletek (nem frissített)

1. A webalkalmazás GitHub elérhetősége: https://github.com/olahsara/jegyzi (Utolsó megtekintés: 2024. 12. 14.)
2. A webalkalmazás hosztolt linkje: https://jegyzi-2024.web.app (Utolsó megtekintés: 2024. 12. 14